NumPy Lab Manual

```
def backward(self, dY):
  self.dW = self.X.T @ dY
  self.db = dY.sum(axis=0)
  return dY @ self.W.T

>>> check_grad(f, g, X)
4.1e-11
```

Matrix calculus for deep learning, coded and checked in NumPy

Part 2 of a three-part course companion to Bishop & Bishop, *Deep Learning: Foundations and Concepts*

1. **Textbook** — derivations by hand, worked examples, exercises with full solutions.
2. **NumPy Lab Manual** (this volume) — every derivative coded and checked numerically; backprop and autodiff from scratch.
3. **Coding Assignments** — six graded assignments with an automatic `pytest` test suite.

Every listing in this manual is the actual source of the accompanying code, and every output box is what that code printed when the manual was built. Every figure was produced by the scripts shown.

Edition of October 2026

About This Manual

The textbook derives derivatives on paper. This manual turns each derivation into a few lines of NumPy and — the habit that matters most — *checks* every one of them numerically. Along the way it builds a small library, `mcalc.py`: a gradient checker, matrix functions, a catalogue of matrix derivatives, a multilayer perceptron with a hand-written backward pass, a forward-mode and a reverse-mode automatic-differentiation engine, and the backward passes of self-attention, layer normalisation and normalizing flows.

The rules

- **NumPy only.** `numpy.linalg` (`solve`, `inv`, `det`, `slogdet`, `cholesky`, `eigh`) is allowed — you implemented those in the SVD course, and here the subject is derivatives.
- **Nothing that differentiates for you** inside `mcalc.py`: no JAX, PyTorch, TensorFlow or autograd. Every gradient is derived by hand.
- Every gradient is **checked** against finite differences. A relative error around 10^{-10} means correct; 10^{-1} means a bug.
- JAX appears in exactly one place: an optional demo in Part 5 that compares our results with `jax.grad`. It is not needed for anything else.

Setting up

You need Python 3.10 or newer and

```
python -m pip install numpy matplotlib pytest
python -m pip install jax          # optional, only for demos/d05_jax.py
```

Unzip `matcalc_lab_code.zip`:

```
lab/
  mcalc.py        the library built in this manual
  plotstyle.py    shared Matplotlib settings
  run_demos.py    runs every demo; output to out/, figures to figs/
  demos/          one script per experiment, d00_shapes.py ... d06_flows.py
```

Run a demo from inside `lab` with `python -m demos.d04_mlp`, or all of them with `python run_demos.py` (about half a minute).

The parts and the course plan

Lab part	Textbook chapters	Assignment
0 NumPy shapes, broadcasting, `einsum`	1	—
1 Gradient checking; matrix functions; identities	1–3, 9	A1
2 Vector derivatives; gradient descent and Newton	4–5	A2
3 Matrix derivatives; Gaussians	6–7	A3
4 Backpropagation: layers and an MLP	8	A4
5 Automatic differentiation; stability; JAX check	9	A5
6 Attention, layer norm, flows, Hutchinson, PCA	10–12	A6

Type the code yourself. Read a section, close the manual, write the function from memory, run the demo, and only then compare. The assignments ask for the same functions — and several the manual does not contain — behind a fixed interface with automatic tests.

Contents

About This Manual i

0 NumPy Shapes, Broadcasting and `einsum` 1

1 Gradient Checking, Matrix Functions and Identities 3

1.1 A gradient checker . . . 3

1.2 Functions of matrices . . . 6

1.3 Matrix identities . . . 9

2 Vector Derivatives and Optimisation 11

2.1 Gradients, Jacobians and Hessians . . . 11

2.2 Gradient descent and Newton's method . . . 13

3 Matrix Derivatives and Gaussians 17

3.1 A catalogue of matrix derivatives . . . 17

3.2 Gaussians . . . 18

4 Backpropagation: Layers and an MLP 23

4.1 Layers with a forward and a backward method . . . 23

4.2 A multilayer perceptron . . . 25

5 Automatic Differentiation and Numerical Stability 29

5.1 Forward mode: dual numbers . . . 29

5.2 Reverse mode: a Tensor that records its history . . . 30

5.3 Numerical stability . . . 35

5.4 Optional: checking against JAX . . . 36

6 Attention, Layer Normalisation and Flows 39

6.1 Self-attention . . . 39

6.2 Layer normalisation . . . 41

6.3 Normalizing flows, Hutchinson and PCA . . . 42

Where Next: the Assignments 46

A Complete Source of `mcalc.py` **47**

B Helper Scripts **62**

CHAPTER 0

NumPy Shapes, Broadcasting and `einsum`

Matrix calculus in code is mostly about shapes. Three NumPy facts are used on every page of this manual.

- **A 1-D array is neither a row nor a column.** `x.shape == (3,)`. `A @ x` treats it as a column, `x @ A` as a row, and `x @ y` is the dot product. Data sets are stored as 2-D arrays with **one example per row**, as in Bishop's design matrices.
- **Broadcasting** stretches axes of length 1 (or missing leading axes) to match: `X - X.mean(axis=0)` subtracts the mean from every row, and `u[:, None] * v[None, :]` is the outer product $\boldsymbol{u}\boldsymbol{v}^\mathsf{T}$.
- **`einsum`** writes index notation (textbook Section 6.6) directly: every index that appears twice is summed. `einsum("i,ij,j->", x, A, x)` is $x_i A_{ij} x_j = \boldsymbol{x}^\mathsf{T} A \boldsymbol{x}$.

demos/d00_shapes.py

```python
import numpy as np

np.set_printoptions(precision=4, suppress=True)
rng = np.random.default_rng(0)
x = np.array([1.0, 2.0, 3.0])          # a vector: shape (3,)
X = rng.standard_normal((4, 3))        # a matrix: 4 rows (examples), 3 columns
    ↪ (features)
print("x.shape =", x.shape, "  X.shape =", X.shape)

# broadcasting: subtract the column means from every row
Xc = X - X.mean(axis=0)
print("column means after centring:", Xc.mean(axis=0))

# outer product u v^T, and a column vector vs a 1-D array
u, v = np.array([1.0, 2.0]), np.array([3.0, 4.0, 5.0])
print("outer(u, v) =\n", np.outer(u, v))
print("u[:, None] * v[None, :] gives the same:", np.allclose(u[:, None] * v[None, :],
    ↪ np.outer(u, v)))

# einsum writes index notation directly: repeated indices are summed
A = rng.standard_normal((3, 3))
print("trace       :", np.einsum("ii->", A), "=", np.trace(A))
print("x^T A x     :", np.einsum("i,ij,j->", x, A, x), "=", x @ A @ x)
print("X A (matmul):", np.allclose(np.einsum("nk,kj->nj", X, A), X @ A))
print("row-wise x_n^T A x_n:", np.einsum("ni,ij,nj->n", X, A, X))

# the gradient of f(W) = a^T W b in index notation: df/dW_kl = a_k b_l
a, b = rng.standard_normal(3), rng.standard_normal(3)
```

```
print("grad of a^T W b equals outer(a, b):", np.allclose(np.einsum("k,l->kl", a, b),
    ↪ np.outer(a, b)))
```

OUTPUT

```
x.shape = (3,)   X.shape = (4, 3)
column means after centring: [ 0. -0.  0.]
outer(u, v) =
 [[ 3.  4.  5.]
 [ 6.  8. 10.]]
u[:, None] * v[None, :] gives the same: True
trace       : -2.9978244204401783 = -2.9978244204401783
x^T A x     : -5.706558664032038 = -5.706558664032035
X A (matmul): True
row-wise x_n^T A x_n: [-0.2118 -0.3174 -5.3983 -4.6599]
grad of a^T W b equals outer(a, b): True
```

The last line is the index-notation derivation of $\partial(\boldsymbol{a}^{\mathsf{T}} W \boldsymbol{b})/\partial W_{kl} = a_k b_l$ written as code. `einsum` is useful for checking an index calculation, but the final formulas in this manual are always plain matrix products, which are easier to read and faster.

CHAPTER 1

Gradient Checking, Matrix Functions and Identities

Textbook: Chapters 1–3 and 9.1–9.2. **Assignment:** A1.

1.1 A gradient checker

The single most useful tool of the course is a function that computes a gradient *numerically*, by central differences (textbook Section 9.1). It works for a scalar function of an array of any shape, and returns an array of the same shape — exactly the rule "the gradient has the shape of the variable".

```
def num_grad(f, X, h=1e-6):
    """Central-difference gradient of a scalar function f at X (any shape).

    Entry i of the result is (f(X + h e_i) - f(X - h e_i)) / 2h, so the
    result has the shape of X -- the rule 'the gradient has the shape of
    the variable'.  Costs 2 * X.size evaluations of f.
    """
    X = np.array(X, dtype=float)                # a private copy we can perturb
    G = np.zeros_like(X)
    for idx in np.ndindex(X.shape):
        old = X[idx]
        X[idx] = old + h
        f_plus = f(X)
        X[idx] = old - h
        f_minus = f(X)
        X[idx] = old                            # restore
        G[idx] = (f_plus - f_minus) / (2 * h)
    return G


def num_jacobian(f, x, h=1e-6):
    """Central-difference Jacobian J[i, j] = d f_i / d x_j of f: R^n -> R^m.

    Inputs and outputs of any shape are flattened, so J is (m, n).
    """
    x = np.array(x, dtype=float)
    m = np.size(f(x))
    J = np.zeros((m, x.size))
    for j, idx in enumerate(np.ndindex(x.shape)):
        old = x[idx]
        x[idx] = old + h
        f_plus = np.ravel(f(x))
        x[idx] = old - h
```

```
        f_minus = np.ravel(f(x))
        x[idx] = old
        J[:, j] = (f_plus - f_minus) / (2 * h)
    return J
```

Two details: we perturb one entry at a time and restore it, and we copy the input first so that the caller's array is never changed. The cost is two evaluations of f per entry, which is fine for test-sized problems and useless for training — that is what the analytic gradient is for.

To compare an analytic gradient with the numerical one we use the **relative error**, which does not depend on the scale of the numbers. The directional check (textbook Section 9.2) needs only two evaluations of f; the complex-step method needs no subtraction at all.

```
def rel_error(a, b, tiny=1e-12):
    """Relative error ||a - b|| / max(||a||, ||b||, tiny) (Frobenius norms)."""
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return float(np.linalg.norm(a - b) / max(np.linalg.norm(a), np.linalg.norm(b), tiny))


def check_grad(f, grad, X, h=1e-6):
    """Relative error between an analytic gradient and central differences."""
    return rel_error(grad, num_grad(f, X, h))


def check_directional(f, grad, X, h=1e-6, seed=0):
    """Cheap check along one random direction v: two evaluations of f.

    Compares (f(X + h v) - f(X - h v)) / 2h with <grad, v>.
    """
    X = np.asarray(X, dtype=float)
    v = np.random.default_rng(seed).standard_normal(X.shape)
    fd = (f(X + h * v) - f(X - h * v)) / (2 * h)
    an = float(np.sum(grad * v))
    return abs(fd - an) / max(abs(fd), abs(an), 1e-12)


def complex_step_grad(f, x, h=1e-20):
    """Complex-step gradient Im f(x + i h e_j) / h: no subtraction, full accuracy.

    Only valid for f built from analytic operations (no abs, max, comparisons).
    """
    x = np.array(x, dtype=complex)
    g = np.zeros(x.shape)
    for idx in np.ndindex(x.shape):
        old = x[idx]
        x[idx] = old + 1j * h
        g[idx] = np.imag(f(x)) / h
        x[idx] = old
    return g
```

demos/d01_gradcheck.py

```python
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, savefig
from mcalc import num_grad, num_jacobian, check_grad, check_directional, complex_step_grad

np.set_printoptions(precision=6, suppress=True)
f = lambda x: x[0] ** 2 * x[1] + 3 * x[1]                    # Worked Example 1.1
print("num_grad at (1,2):", num_grad(f, [1.0, 2.0]), "  exact: [4, 4]")

g = lambda x: np.array([x[0] ** 2 * x[1], 5 * x[0] + np.sin(x[1])])   # Worked Example 1.2
print("num_jacobian at (1,0):\n", num_jacobian(g, [1.0, 0.0]))

# a correct and a wrong gradient for f(x) = x^T A x with non-symmetric A
rng = np.random.default_rng(0)
A = rng.standard_normal((5, 5)); x = rng.standard_normal(5)
q = lambda x: x @ A @ x
print("relative error, (A + A^T) x :", "%.1e" % check_grad(q, (A + A.T) @ x, x))
print("relative error, 2 A x (bug) :", "%.1e" % check_grad(q, 2 * A @ x, x))
print("directional check, correct  :", "%.1e" % check_directional(q, (A + A.T) @ x, x))

# error of three derivative approximations as a function of h, for f(x) = exp(sin x) at x = 1
fs = lambda x: np.exp(np.sin(x))
exact = np.cos(1.0) * np.exp(np.sin(1.0))
hs = np.logspace(-16, -1, 61)
fwd = [abs((fs(1 + h) - fs(1)) / h - exact) for h in hs]
cen = [abs((fs(1 + h) - fs(1 - h)) / (2 * h) - exact) for h in hs]
cpx = [abs(np.imag(fs(1 + 1j * h)) / h - exact) for h in hs]
best_f, best_c = hs[int(np.argmin(fwd))], hs[int(np.argmin(cen))]
print("best h: forward %.0e (error %.1e), central %.0e (error %.1e)"
      % (best_f, min(fwd), best_c, min(cen)))
print("complex step at h=1e-20: error %.1e" % abs(complex_step_grad(lambda v: np.exp(np.sin(v[0])), [1.0], 1e-20)[0] - exact))
plt.loglog(hs, np.maximum(fwd, 1e-17), color=C1, label="forward difference")
plt.loglog(hs, np.maximum(cen, 1e-17), color=C2, label="central difference")
plt.loglog(hs, np.maximum(cpx, 1e-17), color=C3, label="complex step")
plt.xlabel("step $h$"); plt.ylabel("absolute error"); plt.legend(loc="upper center"); savefig("fd_error")
```

OUTPUT

```
num_grad at (1,2): [4. 4.]   exact: [4, 4]
num_jacobian at (1,0):
 [[0. 1.]
 [5. 1.]]
relative error, (A + A^T) x : 4.1e-11
relative error, 2 A x (bug) : 5.1e-01
directional check, correct  : 1.1e-11
best h: forward 1e-08 (error 9.5e-09), central 3e-06 (error 1.1e-11)
complex step at h=1e-20: error 0.0e+00
```

The first two results are textbook Worked Examples 1.1 and 1.2. The next lines show what a bug looks like: the formula $2A\boldsymbol{x}$, which is only correct for symmetric A, has a relative error of 0.5, while the correct $(A + A^{\mathsf{T}})\boldsymbol{x}$ agrees to 10^{-11}. Figure 1.1 plots the error of each approximation against h: the V-shapes are the truncation error (falling with h) meeting the rounding error (rising as h shrinks), with minima where textbook Exercise 9.4 predicts. The complex step has no rounding branch at all.

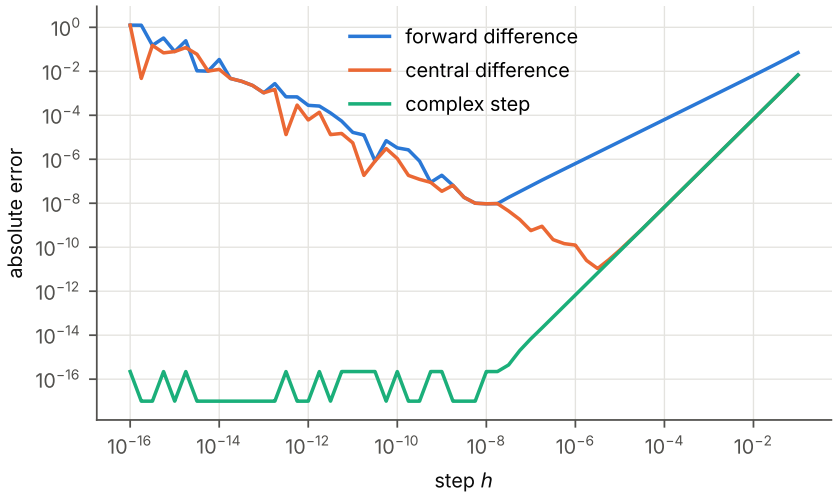


Figure 1.1: Error of three derivative approximations for $f(x) = e^{\sin x}$ at $x = 1$. Forward differences are best at $h \approx 10^{-8}$ (error $\approx 10^{-8}$), central differences at $h \approx 10^{-5}$ (error $\approx 10^{-11}$). The complex step is exact to machine precision for every small h.

> **Key idea.** The default `h=1e-6` in `num_grad` sits near the bottom of the central-difference V. With it, a correct gradient typically gives a relative error of 10^{-9}–10^{-11}, and anything above 10^{-6} deserves suspicion.

1.2 Functions of matrices

```
def logdet_chol(S):
    """ln|S| for symmetric positive definite S, as 2 * sum(log diag(L)), S = L L^T."""
    L = np.linalg.cholesky(S)
    return 2.0 * float(np.sum(np.log(np.diag(L))))


def sym_fn(S, fn):
    """Spectral function f(S) = Q f(Lambda) Q^T of a symmetric matrix."""
    w, Q = np.linalg.eigh(S)
    return (Q * fn(w)) @ Q.T                    # Q diag(f(w)) Q^T without building diag


def sqrtm_sym(S):
    return sym_fn(S, lambda w: np.sqrt(np.clip(w, 0.0, None)))


```

```
def inv_sqrtm_sym(S):
    return sym_fn(S, lambda w: 1.0 / np.sqrt(w))


def expm_sym(S):
    return sym_fn(S, np.exp)


def logm_sym(S):
    return sym_fn(S, np.log)


def expm_taylor(A, terms=20):
    """exp(A) for any square A: Taylor series with scaling and squaring.

    exp(A) = exp(A / 2^s)^(2^s); choosing s so that ||A / 2^s|| < 1/2 makes
    the series converge fast, then s squarings undo the scaling.
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    norm1 = np.max(np.sum(np.abs(A), axis=0)) if A.size else 0.0
    s = max(0, int(np.ceil(np.log2(norm1))) + 1) if norm1 > 0 else 0
    B = A / 2.0 ** s
    E, term = np.eye(n), np.eye(n)
    for k in range(1, terms + 1):
        term = term @ B / k
        E = E + term
    for _ in range(s):
        E = E @ E
    return E
```

`sym_fn` is the spectral definition of textbook Section 2.4 in one line: `(Q * fn(w)) @ Q.T` scales the columns of Q by $f(\lambda_i)$, which equals $Q\,\mathrm{diag}(f(\boldsymbol{\lambda}))Q^{\mathsf{T}}$ without building the diagonal matrix. For non-symmetric matrices, `expm_taylor` uses the power series with *scaling and squaring*: the series converges fast for a matrix of small norm, and $\exp(A) = \exp(A/2^s)^{2^s}$ undoes the scaling.

demos/d01_matfuncs.py

```
import numpy as np
from mcalc import logdet_chol, sqrtm_sym, inv_sqrtm_sym, expm_sym, expm_taylor

np.set_printoptions(precision=4, suppress=True)
S = np.array([[4.0, 2.0], [2.0, 3.0]])                          # Worked Example 2.1
print("trace %.0f, det %.0f, eigenvalues %s" % (np.trace(S), np.linalg.det(S),
    ↪ np.linalg.eigvalsh(S)))
print("Cholesky L =\n", np.linalg.cholesky(S))
print("logdet via Cholesky %.6f   ln 8 = %.6f" % (logdet_chol(S), np.log(8)))

# determinants underflow; log-determinants do not
B = 0.1 * np.eye(400)
print("det(0.1 I_400)          =", np.linalg.det(B))
print("log(det(0.1 I_400))     =", np.log(np.linalg.det(B)) if np.linalg.det(B) > 0 else
    ↪ "-inf (log of 0)")
print("logdet_chol(0.1 I_400)  = %.4f   (400 ln 0.1 = %.4f)" % (logdet_chol(B), 400 *
```

```
    ↪ np.log(0.1)))

R = sqrtm_sym(np.array([[5.0, 4.0], [4.0, 5.0]]))              # Worked Example 2.2
print("sqrt([[5,4],[4,5]]) =\n", R, "\nsquared =\n", R @ R)
W = inv_sqrtm_sym(np.array([[5.0, 4.0], [4.0, 5.0]]))
print("3 * S^(-1/2) =\n", 3 * W)

E = expm_sym(np.array([[0.0, 1.0], [1.0, 0.0]]))               # Worked Example 2.3
print("exp([[0,1],[1,0]]) =\n", E, "\ncosh 1 = %.4f, sinh 1 = %.4f" % (np.cosh(1),
    ↪ np.sinh(1)))
th = 0.7
Rot = expm_taylor(np.array([[0.0, -th], [th, 0.0]]))
print("exp of skew matrix (theta=0.7) =\n", Rot, "\ncos 0.7 = %.4f, sin 0.7 = %.4f" %
    ↪ (np.cos(th), np.sin(th)))
M = np.random.default_rng(1).standard_normal((4, 4))
print("det(exp(M)) = %.6f   exp(tr M) = %.6f" % (np.linalg.det(expm_taylor(M)),
    ↪ np.exp(np.trace(M))))
```

OUTPUT

```
trace 7, det 8, eigenvalues [1.4384 5.5616]
Cholesky L =
 [[2.     0.    ]
 [1.     1.4142]]
logdet via Cholesky 2.079442   ln 8 = 2.079442
det(0.1 I_400)          = 0.0
log(det(0.1 I_400))     = -inf (log of 0)
logdet_chol(0.1 I_400)  = -921.0340   (400 ln 0.1 = -921.0340)
sqrt([[5,4],[4,5]]) =
 [[2. 1.]
 [1. 2.]]
squared =
 [[5. 4.]
 [4. 5.]]
3 * S^(-1/2) =
 [[ 2. -1.]
 [-1.  2.]]
exp([[0,1],[1,0]]) =
 [[1.5431 1.1752]
 [1.1752 1.5431]]
cosh 1 = 1.5431, sinh 1 = 1.1752
exp of skew matrix (theta=0.7) =
 [[ 0.7648 -0.6442]
 [ 0.6442  0.7648]]
cos 0.7 = 0.7648, sin 0.7 = 0.6442
det(exp(M)) = 4.133925   exp(tr M) = 4.133925
```

The numbers are textbook Worked Examples 2.1–2.3 and Exercise 2.4. The determinant of $0.1 I_{400}$ is 10^{-400}, below the smallest double, so `det` returns exactly 0 and its logarithm is $-\infty$; the Cholesky log-determinant gives the correct $400 \ln 0.1$. The last line confirms $\det \exp(M) = e^{\operatorname{tr} M}$ for a random 4×4 matrix.

1.3 Matrix identities

```
def sherman_morrison(Ainv, u, v):
    """(A + u v^T)^{-1} from A^{-1} in O(n^2)."""
    Au = Ainv @ u
    vA = v @ Ainv
    return Ainv - np.outer(Au, vA) / (1.0 + v @ Au)


def woodbury(Ainv, U, C, V):
    """(A + U C V)^{-1} from A^{-1}: only a k x k system is solved."""
    K = np.linalg.inv(C) + V @ Ainv @ U          # k x k
    return Ainv - Ainv @ U @ np.linalg.solve(K, V @ Ainv)


def block_inverse(A, B, C, D):
    """Inverse of [[A, B], [C, D]] via the Schur complement of D."""
    Dinv = np.linalg.inv(D)
    P = np.linalg.inv(A - B @ Dinv @ C)          # (M/D)^{-1}
    top = np.hstack([P, -P @ B @ Dinv])
    bottom = np.hstack([-Dinv @ C @ P, Dinv + Dinv @ C @ P @ B @ Dinv])
    return np.vstack([top, bottom])
```

demos/d01_identities.py

```
import time
import numpy as np
from mcalc import sherman_morrison, woodbury, block_inverse

np.set_printoptions(precision=4, suppress=True)
A = np.diag([2.0, 4.0]); u = v = np.array([1.0, 1.0])          # Worked Example 3.3
print("14 * (A + u u^T)^-1 =\n", 14 * sherman_morrison(np.linalg.inv(A), u, v))
print("determinant lemma: %.4f vs det %.4f" % ((1 + v @ np.linalg.solve(A, u)) *
    ↪ np.linalg.det(A),
                                              np.linalg.det(A + np.outer(u, v))))

M = np.array([[2.0, 1, 0], [1, 2, 1], [0, 1, 2]])               # Worked Example 3.2
Minv = block_inverse(M[:1, :1], M[:1, 1:], M[1:, :1], M[1:, 1:])
print("4 * M^-1 by blocks =\n", 4 * Minv, "\nmatches inv:", np.allclose(Minv,
    ↪ np.linalg.inv(M)))

# Woodbury: a diagonal matrix plus a rank-k update, n = 2000, k = 10
rng = np.random.default_rng(0)
n, k = 2000, 10
d = rng.uniform(1, 2, n); U = rng.standard_normal((n, k)); C = np.eye(k)
Ainv = np.diag(1 / d)
t0 = time.perf_counter(); direct = np.linalg.inv(np.diag(d) + U @ C @ U.T); t1 =
    ↪ time.perf_counter()
wood = woodbury(Ainv, U, C, U.T); t2 = time.perf_counter()
print("n=%d, k=%d: direct inverse %.3f s, Woodbury %.3f s, max difference %.1e"
      % (n, k, t1 - t0, t2 - t1, np.abs(direct - wood).max()))
```

OUTPUT

```
14 * (A + u u^T)^-1 =
 [[ 5. -1.]
 [-1.  3.]]
determinant lemma: 14.0000 vs det 14.0000
4 * M^-1 by blocks =
 [[ 3. -2.  1.]
 [-2.  4. -2.]
 [ 1. -2.  3.]]
matches inv: True
n=2000, k=10: direct inverse 0.673 s, Woodbury 0.048 s, max difference 7.1e-14
```

Sherman–Morrison and the partitioned inverse reproduce textbook Worked Examples 3.3 and 3.2. The Woodbury timing shows the point of the identity: for a 2000×2000 diagonal matrix plus a rank-10 update, it is an order of magnitude faster than a direct inverse and agrees to 10^{-13}.

Try it yourself.

1. Use `check_grad` on $f(\boldsymbol{x}) = \|\boldsymbol{x}\|$ at $\boldsymbol{x} = (3, 4)$ with the gradient $\boldsymbol{x}/\|\boldsymbol{x}\|$. Then try the point $\boldsymbol{x} = (0, 0)$: what happens, and why?
2. Plot the error of `expm_taylor` against the number of terms, with and without scaling and squaring, for $A = 10 \cdot \left[\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right]$.

CHAPTER 2

Vector Derivatives and Optimisation

Textbook: Chapters 4–5. **Assignment:** A2.

2.1 Gradients, Jacobians and Hessians

Each function returns its value together with its gradient (and Hessian where useful). Returning them together is the pattern of every optimiser: one call gives everything a step needs.

```
def quadratic(x, A, b):
    """f = x^T A x + b^T x with gradient (A + A^T) x + b and Hessian A + A^T."""
    f = x @ A @ x + b @ x
    return f, (A + A.T) @ x + b, A + A.T


def least_squares(w, Phi, t, lam=0.0):
    """E = 1/2 ||Phi w - t||^2 + lam/2 ||w||^2, its gradient and Hessian."""
    r = Phi @ w - t
    E = 0.5 * r @ r + 0.5 * lam * w @ w
    g = Phi.T @ r + lam * w
    H = Phi.T @ Phi + lam * np.eye(len(w))
    return E, g, H
```

The numerically stable building blocks come next (textbook Section 9.5). Note how `sigmoid` never evaluates e^x for positive x: it computes $e^{-|a|} \leq 1$ and assembles both branches from it.

```
def logsumexp(a, axis=-1, keepdims=False):
    """Stable log(sum(exp(a))): subtract the maximum first."""
    a = np.asarray(a, dtype=float)
    m = np.max(a, axis=axis, keepdims=True)
    out = m + np.log(np.sum(np.exp(a - m), axis=axis, keepdims=True))
    return out if keepdims else np.squeeze(out, axis=axis)


def softmax(a, axis=-1):
    """Stable softmax along an axis."""
    a = np.asarray(a, dtype=float)
    e = np.exp(a - np.max(a, axis=axis, keepdims=True))
    return e / np.sum(e, axis=axis, keepdims=True)


def softplus(a):
    """log(1 + exp(a)) without overflow: max(a, 0) + log1p(exp(-|a|))."""
    a = np.asarray(a, dtype=float)
    return np.maximum(a, 0.0) + np.log1p(np.exp(-np.abs(a)))

```

```
225
226 def sigmoid(a):
227     """Logistic sigmoid, evaluated so that exp never overflows."""
228     a = np.asarray(a, dtype=float)
229     e = np.exp(-np.abs(a))                      # always <= 1
230     return np.where(a >= 0, 1.0 / (1.0 + e), e / (1.0 + e))
231
232
233 def log_sigmoid(a):
234     return -softplus(-np.asarray(a, dtype=float))
```

```
239 def softmax_jacobian(a):
240     """d softmax(a) / d a = diag(s) - s s^T."""
241     s = softmax(a)
242     return np.diag(s) - np.outer(s, s)
243
244
245 def logistic_loss(w, Phi, t):
246     """Cross-entropy of logistic regression: value, gradient Phi^T (y - t), Hessian
    ↪ Phi^T R Phi.
247
248     The value is computed as sum(softplus(a) - t a), which never takes log(0).
249     """
250     a = Phi @ w
251     E = float(np.sum(softplus(a) - t * a))
252     y = sigmoid(a)
253     g = Phi.T @ (y - t)
254     H = Phi.T @ (Phi * (y * (1 - y))[:, None])
255     return E, g, H
```

The logistic loss is computed as $\sum_n \operatorname{softplus}(a_n) - t_n a_n$, which is algebraically equal to the cross-entropy but never takes the logarithm of a probability that has underflowed to 0.

demos/d02_vector.py

```
1  import numpy as np
2  from mcalc import (quadratic, least_squares, softmax, softmax_jacobian, logsumexp,
3                     logistic_loss, num_grad, num_jacobian, check_grad, rel_error)
4
5  np.set_printoptions(precision=4, suppress=True)
6  A = np.array([[2.0, 1.0], [1.0, 3.0]]); b = np.array([1.0, -1.0])     # Worked Example
       ↪ 4.1
7  f, g, H = quadratic(np.array([1.0, 2.0]), A, b)
8  print("f = %.1f, gradient = %s, Hessian eigenvalues = %s" % (f, g,
       ↪ np.linalg.eigvalsh(H)))
9  print("minimiser -A^{-1} b / 2 =", -0.5 * np.linalg.solve(A, b))
10
11 a = np.array([0.0, np.log(2.0), 0.0])                                 # Worked Example
       ↪ 4.2
12 print("softmax =", softmax(a))
13 print("16 * Jacobian =\n", 16 * softmax_jacobian(a))
14 print("Jacobian vs finite differences: %.1e" % rel_error(softmax_jacobian(a),
       ↪ num_jacobian(softmax, a)))
15 print("grad of logsumexp equals softmax: %.1e" % rel_error(num_grad(logsumexp, a),
```

```
    ↪ softmax(a)))

Phi = np.array([[1.0, 0.0], [1.0, 1.0], [1.0, 2.0]]); t = np.array([6.0, 0.0, 0.0])
w = np.linalg.solve(Phi.T @ Phi, Phi.T @ t)                          # normal equations
print("line fit w =", w, "  gradient there:", least_squares(w, Phi, t)[1])

rng = np.random.default_rng(0)
Phi = rng.standard_normal((20, 3)); tt = (rng.random(20) < 0.5).astype(float); w =
    ↪ rng.standard_normal(3)
E, g, H = logistic_loss(w, Phi, tt)
print("logistic loss: gradient error %.1e, Hessian error %.1e"
      % (check_grad(lambda v: logistic_loss(v, Phi, tt)[0], g, w),
         rel_error(H, num_jacobian(lambda v: logistic_loss(v, Phi, tt)[1], w))))
print("Hessian eigenvalues (all >= 0, convex):", np.linalg.eigvalsh(H))
```

OUTPUT

```
f = 17.0, gradient = [ 9. 13.], Hessian eigenvalues = [2.7639 7.2361]
minimiser -A^{-1} b / 2 = [-0.4  0.3]
softmax = [0.25 0.5  0.25]
16 * Jacobian =
 [[ 3. -2. -1.]
 [-2.  4. -2.]
 [-1. -2.  3.]]
Jacobian vs finite differences: 1.1e-10
grad of logsumexp equals softmax: 1.4e-10
line fit w = [ 5. -3.]   gradient there: [0. 0.]
logistic loss: gradient error 6.9e-10, Hessian error 1.5e-10
Hessian eigenvalues (all >= 0, convex): [0.9895 1.4419 4.3498]
```

The results reproduce textbook Worked Examples 4.1 and 4.2 and the line fit of Section 4.2. The last two lines verify the logistic-regression gradient $\Phi^{\mathsf{T}}(\boldsymbol{y} - \boldsymbol{t})$ and Hessian $\Phi^{\mathsf{T}} R \Phi$ of textbook Exercise 4.7 numerically: the Hessian is checked by taking the numerical Jacobian of the analytic gradient.

2.2 Gradient descent and Newton's method

```
def gradient_descent(grad_fn, x0, lr, n_steps):
    """Plain gradient descent; returns the array of iterates x_0, ..., x_n."""
    xs = [np.array(x0, dtype=float)]
    for _ in range(n_steps):
        xs.append(xs[-1] - lr * grad_fn(xs[-1]))
    return np.array(xs)


def newton(grad_fn, hess_fn, x0, n_steps):
    """Newton's method x <- x - H^{-1} g (a solve, never an inverse)."""
    xs = [np.array(x0, dtype=float)]
    for _ in range(n_steps):
        x = xs[-1]
        xs.append(x - np.linalg.solve(np.atleast_2d(hess_fn(x)),
    ↪ np.atleast_1d(grad_fn(x))))
    return np.array(xs)
```

demos/d02_optim.py

```python
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, INK2, savefig
from mcalc import gradient_descent, newton, logistic_loss, sigmoid

np.set_printoptions(precision=4, suppress=True)
H = np.diag([1.0, 10.0]); grad = lambda w: H @ w                        # Worked Example 5.1
for lr in (0.1, 0.19, 0.21):
    W = gradient_descent(grad, [1.0, 1.0], lr, 3)
    print("lr=%.2f  iterates:" % lr, " ".join("(%.3f, %.3f)" % tuple(w) for w in W))

fig, ax = plt.subplots(1, 2, figsize=(7.4, 3.0))
g1, g2 = np.meshgrid(np.linspace(-1.2, 1.2, 200), np.linspace(-1.2, 1.2, 200))
ax[0].contour(g1, g2, 0.5 * (g1 ** 2 + 10 * g2 ** 2), levels=12, colors="#c9c7c0", linewidths=0.6)
for lr, c in zip((0.05, 0.18), (C1, C2)):
    W = gradient_descent(grad, [-1.0, 1.0], lr, 25)
    ax[0].plot(W[:, 0], W[:, 1], "o-", ms=2.5, color=c, label="$\\eta=%.2f$" % lr)
ax[0].set_aspect("equal"); ax[0].legend(fontsize=7); ax[0].set_title("paths on $H=$diag$(1,10)$", fontsize=8)
for kappa, c in zip((2, 10, 100), (C1, C2, C3)):
    Hk = np.diag([1.0, kappa])
    lr = 2 / (1 + kappa)
    W = gradient_descent(lambda w: Hk @ w, [1.0, 1.0], lr, 60)
    ax[1].semilogy(np.linalg.norm(W, axis=1), color=c, label="$\\kappa=%d$" % kappa)
ax[1].set_xlabel("iteration"); ax[1].set_ylabel("$\\|w_k-w^\\star\\|$"); ax[1].legend(fontsize=7)
ax[1].set_title("optimal step, three condition numbers", fontsize=8)
savefig("gd_paths")

# logistic regression: Newton (IRLS) versus gradient descent
x = np.array([1.0, 2.0, -1.0, -2.0]); t = np.array([1.0, 0.0, 0.0, 1.0])   # Worked Example 5.2
Phi = x[:, None]
g = lambda w: logistic_loss(w, Phi, t)[1]; h = lambda w: logistic_loss(w, Phi, t)[2]
print("Newton iterates:", newton(g, h, [0.0], 4).ravel())

rng = np.random.default_rng(0)
N = 200
Xd = rng.standard_normal((N, 2)) * [1.0, 4.0]                       # badly scaled features
Phi = np.column_stack([np.ones(N), Xd])
w_true = np.array([0.5, 2.0, -0.5])
t = (rng.random(N) < sigmoid(Phi @ w_true)).astype(float)
Wn = newton(lambda w: logistic_loss(w, Phi, t)[1], lambda w: logistic_loss(w, Phi, t)[2], np.zeros(3), 8)
Wg = gradient_descent(lambda w: logistic_loss(w, Phi, t)[1] / N, np.zeros(3), 0.5, 2000)
w_star = Wn[-1]
print("Newton: |w_k - w*| =", " ".join("%.1e" % np.linalg.norm(w - w_star) for w in Wn[:7]))
```

```python
for k in (10, 100, 1000, 2000):
    print("GD after %4d steps: |w - w*| = %.1e" % (k, np.linalg.norm(Wg[k] - w_star)))
```

OUTPUT

```
lr=0.10  iterates: (1.000, 1.000) (0.900, 0.000) (0.810, 0.000) (0.729, 0.000)
lr=0.19  iterates: (1.000, 1.000) (0.810, -0.900) (0.656, 0.810) (0.531, -0.729)
lr=0.21  iterates: (1.000, 1.000) (0.790, -1.100) (0.624, 1.210) (0.493, -1.331)
Newton iterates: [ 0.     -0.4    -0.4195 -0.4196 -0.4196]
Newton: |w_k - w*| = 2.4e+00 1.3e+00 5.7e-01 1.4e-01 8.3e-03 3.1e-05 4.2e-10
GD after   10 steps: |w - w*| = 1.4e+00
GD after  100 steps: |w - w*| = 1.4e-01
GD after 1000 steps: |w - w*| = 1.6e-09
GD after 2000 steps: |w - w*| = 1.1e-14
```

The first three lines are textbook Worked Example 5.1: with $\eta = 0.1$ the steep coordinate is solved in one step, with $\eta = 0.19$ it oscillates, and with $\eta = 0.21 > 2/\lambda_{\max}$ it grows by 10% per step. Figure 2.1 shows the zig-zag and the effect of the condition number. The Newton iterates continue Worked Example 5.2: w moves from -0.4 to -0.4196 in two more steps and stays there — exactly the small move to the left that textbook Exercise 5.5 predicted.

The second experiment is logistic regression on 200 points with badly scaled features. Newton's method converges *quadratically*: the error goes $1.4 \times 10^{-1} \to 8 \times 10^{-3} \to 3 \times 10^{-5} \to 4 \times 10^{-10}$, roughly doubling the number of correct digits each step. Gradient descent needs about a thousand steps for the same accuracy.

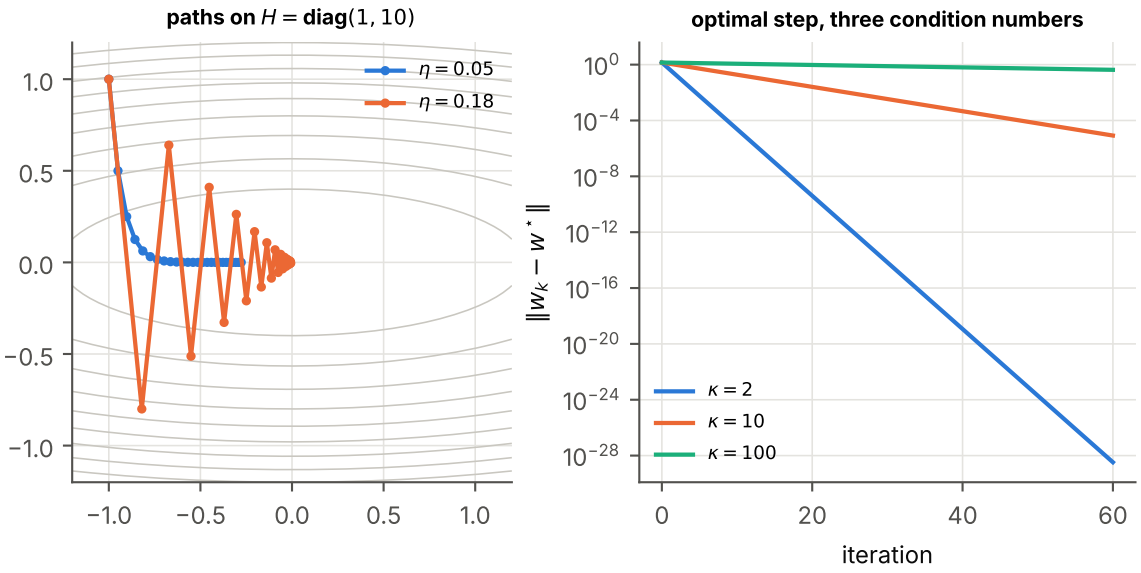


Figure 2.1: Left: gradient descent on $E = \frac{1}{2}(w_1^2 + 10w_2^2)$ with a small step (smooth but slow) and a large one (zig-zag across the valley). Right: error per iteration at the optimal step size for $\kappa = 2, 10, 100$; the rate $\frac{\kappa-1}{\kappa+1}$ approaches 1 as κ grows.

Try it yourself.

1. Standardise the two features of the logistic-regression experiment (zero mean, unit variance) and count again how many gradient-descent steps are needed. Compute the condition number of the Hessian before and after.
2. Add momentum to `gradient_descent` and reproduce the $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ rate for $\kappa = 100$.

CHAPTER 3

Matrix Derivatives and Gaussians

Textbook: Chapters 6–7. **Assignment:** A3.

3.1 A catalogue of matrix derivatives

Theorem 6.3 of the textbook, as a dictionary of (function, gradient) pairs. Each gradient is the formula derived with the differential method; the test is whether it matches finite differences.

```
# Each entry: name -> (f(X, P), grad(X, P)), where P holds the constants.
CATALOGUE = {
    "tr(A X)":          (lambda X, P: np.trace(P["A"] @ X),
                         lambda X, P: P["A"].T),
    "tr(A X B)":        (lambda X, P: np.trace(P["A"] @ X @ P["B"]),
                         lambda X, P: P["A"].T @ P["B"].T),
    "tr(X^T A X)":      (lambda X, P: np.trace(X.T @ P["A"] @ X),
                         lambda X, P: (P["A"] + P["A"].T) @ X),
    "a^T X b":          (lambda X, P: P["a"] @ X @ P["b"],
                         lambda X, P: np.outer(P["a"], P["b"])),
    "ln|det X|":        (lambda X, P: np.linalg.slogdet(X)[1],
                         lambda X, P: np.linalg.inv(X).T),
    "det X":            (lambda X, P: np.linalg.det(X),
                         lambda X, P: np.linalg.det(X) * np.linalg.inv(X).T),
    "tr(X^-1 A)":       (lambda X, P: np.trace(np.linalg.solve(X, P["A"])),
                         lambda X, P: -np.linalg.inv(X).T @ P["A"].T @
    ↪ np.linalg.inv(X).T),
    "a^T X^-1 b":       (lambda X, P: P["a"] @ np.linalg.solve(X, P["b"]),
                         lambda X, P: -np.linalg.inv(X).T @ np.outer(P["a"], P["b"]) @
    ↪ np.linalg.inv(X).T),
    "||A X - B||_F^2":  (lambda X, P: np.sum((P["A"] @ X - P["B"]) ** 2),
                         lambda X, P: 2 * P["A"].T @ (P["A"] @ X - P["B"])),
    "tr(A^T tanh(X))":  (lambda X, P: np.sum(P["A"] * np.tanh(X)),
                         lambda X, P: P["A"] * (1 - np.tanh(X) ** 2)),
}
```

demos/d03_catalogue.py

```
import numpy as np
from mcalc import CATALOGUE, num_grad, rel_error

rng = np.random.default_rng(0)
n = 4
P = {"A": rng.standard_normal((n, n)), "B": rng.standard_normal((n, n)),
     "a": rng.standard_normal(n), "b": rng.standard_normal(n)}
X = rng.standard_normal((n, n)) + 2 * np.eye(n)        # comfortably invertible
print("%-18s %12s" % ("function", "rel. error"))
for name, (f, grad) in CATALOGUE.items():
```

```
    err = rel_error(grad(X, P), num_grad(lambda Y: f(Y, P), X))
    print("%-18s %12.1e" % (name, err))

# Worked Examples 6.1 and 6.2
X1 = np.array([[2.0, 1.0], [1.0, 1.0]])
print("d ln|X| / dX =\n", CATALOGUE["ln|det X|"][1](X1, {}))
A2 = np.array([[1.0, 2.0], [0.0, 1.0]]); X2 = np.array([[1.0, 0.0], [1.0, 1.0]])
print("d tr(X^T A X) / dX =\n", CATALOGUE["tr(X^T A X)"][1](X2, {"A": A2}))
```

OUTPUT

```
function             rel. error
tr(A X)                 1.8e-10
tr(A X B)               4.9e-10
tr(X^T A X)             1.8e-10
a^T X b                 2.6e-10
ln|det X|               1.3e-10
det X                   1.2e-10
tr(X^-1 A)              1.2e-10
a^T X^-1 b              1.4e-10
||A X - B||_F^2         4.8e-10
tr(A^T tanh(X))         1.7e-10
d ln|X| / dX =
 [[ 1. -1.]
 [-1.  2.]]
d tr(X^T A X) / dX =
 [[4. 2.]
 [4. 2.]]
```

Every entry agrees with finite differences to about 10^{-10}, and the last two outputs are textbook Worked Examples 6.1 and 6.2. Notice that the functions use `solve` and `slogdet` for their *values*; only the gradient formulas contain `inv`, because the formula itself contains X^{-1}.

3.2 Gaussians

```
def gaussian_nll(X, mu, Sigma):
    """Negative log-likelihood of the rows of X under N(mu, Sigma), with gradients.

    Returns (value, d/dmu, d/dSigma) where
        d/dmu    = -Sigma^{-1} sum_n (x_n - mu)
        d/dSigma = 1/2 (N Sigma^{-1} - Sigma^{-1} S Sigma^{-1}),  S = sum
    ↪ (x_n-mu)(x_n-mu)^T
    """
    X = np.atleast_2d(X)
    N, D = X.shape
    diff = X - mu
    Sinv = np.linalg.inv(Sigma)
    S = diff.T @ diff
    # slogdet, not logdet_chol: Cholesky reads only the lower triangle, so a finite
    # difference that perturbs an upper entry would see a different function
    value = 0.5 * N * D * np.log(2 * np.pi) + 0.5 * N * np.linalg.slogdet(Sigma)[1] +
    ↪ 0.5 * np.sum(Sinv * S)
    dmu = -Sinv @ diff.sum(axis=0)
```

```
    dSigma = 0.5 * (N * Sinv - Sinv @ S @ Sinv)
    return float(value), dmu, dSigma


def gaussian_mle(X):
    """Maximum-likelihood mean and covariance (divide by N, as in Bishop)."""
    X = np.atleast_2d(X)
    mu = X.mean(axis=0)
    diff = X - mu
    return mu, diff.T @ diff / X.shape[0]


def fit_gaussian_gd(X, lr=0.1, steps=500):
    """Fit N(mu, L L^T) by gradient descent on (mu, L), L lower triangular.

    Chain rule through Sigma = L L^T:  dE/dL = 2 (dE/dSigma) L, keeping the
    lower triangle.  Returns mu, Sigma and the loss history (NLL per point).
    """
    X = np.atleast_2d(X)
    N, D = X.shape
    mu, L = np.zeros(D), np.eye(D)
    history = []
    for _ in range(steps):
        E, dmu, dSigma = gaussian_nll(X, mu, L @ L.T)
        history.append(E / N)
        mu = mu - lr * dmu / N
        L = L - lr * np.tril(2 * dSigma @ L) / N
    return mu, L @ L.T, np.array(history)


def kl_diag_standard(mu, logvar):
    """KL( N(mu, diag(exp(logvar))) || N(0, I) ) and its gradients (the VAE term)."""
    var = np.exp(logvar)
    value = 0.5 * float(np.sum(var + mu ** 2 - 1.0 - logvar))
    return value, mu.copy(), 0.5 * (var - 1.0)
```

A bug we made while writing this manual. Our first version of `gaussian_nll` computed $\ln|\Sigma|$ with the Cholesky factor. The gradient check for Σ then failed with a relative error of 0.27, although the formula was right. The reason: `np.linalg.cholesky` reads only the *lower* triangle of its argument. A finite difference that perturbs an entry above the diagonal makes Σ non-symmetric, which Cholesky silently ignores — so the numerical gradient was the gradient of a different function. With `slogdet`, which uses the whole matrix, the check passes. Lesson: the function you check must be a function of *every* entry you perturb.

`fit_gaussian_gd` fits a covariance by gradient descent. Optimising Σ directly could leave the set of positive definite matrices, so it optimises a lower-triangular factor L with $\Sigma = LL^{\mathsf{T}}$. The chain rule through $\Sigma = LL^{\mathsf{T}}$ is one application of the differential method: $\mathrm{d}\Sigma = \mathrm{d}L\,L^{\mathsf{T}} + L\,\mathrm{d}L^{\mathsf{T}}$, so $\mathrm{d}E = \operatorname{tr}(G^{\mathsf{T}}\mathrm{d}\Sigma) = 2\operatorname{tr}(L^{\mathsf{T}}G\,\mathrm{d}L)$ for symmetric $G = \partial E/\partial\Sigma$, and $\partial E/\partial L = 2GL$ (keeping the lower triangle).

demos/d03_gaussian.py

```python
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, INK2, savefig
from mcalc import gaussian_nll, gaussian_mle, fit_gaussian_gd, kl_diag_standard, num_grad, rel_error

np.set_printoptions(precision=4, suppress=True)
X = np.array([[1.0, 2.0], [3.0, 4.0], [2.0, 0.0]])                # Worked Example 7.1
mu, Sigma = gaussian_mle(X)
print("mu_ML =", mu, "\n3 * Sigma_ML =\n", 3 * Sigma)
_, dmu, dSigma = gaussian_nll(X, mu, Sigma)
print("gradients at the MLE: |dmu| = %.1e, |dSigma| = %.1e" % (np.abs(dmu).max(), np.abs(dSigma).max()))

rng = np.random.default_rng(0)
Xr = rng.standard_normal((50, 3)) @ np.array([[1, 0, 0], [0.5, 1, 0], [0.2, 0.3, 0.5]])
m0 = rng.standard_normal(3); S0 = np.cov(rng.standard_normal((3, 20)))
_, dmu, dS = gaussian_nll(Xr, m0, S0)
print("gradient check: dmu %.1e, dSigma %.1e"
      % (rel_error(dmu, num_grad(lambda m: gaussian_nll(Xr, m, S0)[0], m0)),
         rel_error(dS, num_grad(lambda S: gaussian_nll(Xr, m0, S)[0], S0))))

# fit a correlated 2-D Gaussian by gradient descent on (mu, L)
C = np.array([[2.0, 1.2], [1.2, 1.0]])
D2 = rng.multivariate_normal([1.0, -1.0], C, size=400)
mu_gd, Sig_gd, hist = fit_gaussian_gd(D2, lr=0.2, steps=400)
mu_ml, Sig_ml = gaussian_mle(D2)
print("gradient descent: mu =", mu_gd, " Sigma =\n", Sig_gd)
print("closed form     : mu =", mu_ml, " Sigma =\n", Sig_ml)
fig, ax = plt.subplots(1, 2, figsize=(7.2, 3.0))
ax[0].semilogx(np.arange(1, len(hist) + 1), hist, color=C1); ax[0].set_xlabel("step"); ax[0].set_ylabel("NLL per point")
ax[1].scatter(D2[:, 0], D2[:, 1], s=4, color="#9a9890")
g1, g2 = np.meshgrid(np.linspace(-3, 5, 200), np.linspace(-4, 2, 200))
G = np.stack([g1 - mu_gd[0], g2 - mu_gd[1]], axis=-1)
q = np.einsum("...i,ij,...j->...", G, np.linalg.inv(Sig_gd), G)
ax[1].contour(g1, g2, q, levels=[1, 4, 9], colors=C2, linewidths=1.2)
ax[1].set_aspect("equal"); ax[1].set_title("1, 2, 3 standard-deviation contours", fontsize=8)
savefig("gauss_fit")

# the VAE KL term (Worked Example 7.3) and the reparameterisation trick
val, dmu, dlv = kl_diag_standard(np.array([1.0, 0.0]), np.log(np.array([1.0, 4.0])))
print("KL = %.4f, dKL/dmu = %s, dKL/dlogvar = %s" % (val, dmu, dlv))
mu = np.array([0.5, -1.0]); L = np.array([[1.0, 0.0], [0.6, 0.8]])
eps = rng.standard_normal((200000, 2)); Z = mu + eps @ L.T            # z = mu + L eps
grad_f = 2 * Z                                                       # f(z) = ||z||^2
print("reparameterisation, d/dmu E||z||^2: MC %s, exact %s" % (grad_f.mean(0), 2 * mu))
print("d/dL E||z||^2: MC\n", np.tril(grad_f.T @ eps / len(eps)), "\nexact 2L (lower part)\n", 2 * L)
```

OUTPUT

```
mu_ML = [2. 2.]
3 * Sigma_ML =
 [[2. 2.]
 [2. 8.]]
gradients at the MLE: |dmu| = 0.0e+00, |dSigma| = 0.0e+00
gradient check: dmu 7.3e-11, dSigma 6.5e-10
gradient descent: mu = [ 1.078  -0.9964]  Sigma =
 [[1.8531 1.0917]
 [1.0917 0.914 ]]
closed form     : mu = [ 1.078  -0.9964]  Sigma =
 [[1.8531 1.0917]
 [1.0917 0.914 ]]
KL = 1.3069, dKL/dmu = [1. 0.], dKL/dlogvar = [0.  1.5]
reparameterisation, d/dmu E||z||^2: MC [ 0.9979 -1.9987], exact [ 1. -2.]
d/dL E||z||^2: MC
 [[2.0035 0.    ]
 [1.2035 1.6019]]
exact 2L (lower part)
 [[2.  0. ]
 [1.2 1.6]]
```

The output reproduces textbook Worked Examples 7.1 and 7.3. Gradient descent on $(\boldsymbol{\mu}, L)$ converges to exactly the closed-form maximum-likelihood estimates (Figure 3.1). The last lines check the reparameterisation gradients of textbook Section 7.5 by Monte Carlo: for $f(\boldsymbol{z}) = \|\boldsymbol{z}\|^2$, $\mathbb{E}f = \|\boldsymbol{\mu}\|^2 + \operatorname{tr}(LL^\mathsf{T})$, whose exact gradients are $2\boldsymbol{\mu}$ and $2L$; averages over 200,000 samples of ∇f and $\nabla f\, \boldsymbol{\epsilon}^\mathsf{T}$ agree to about two decimals, as Monte Carlo error predicts.

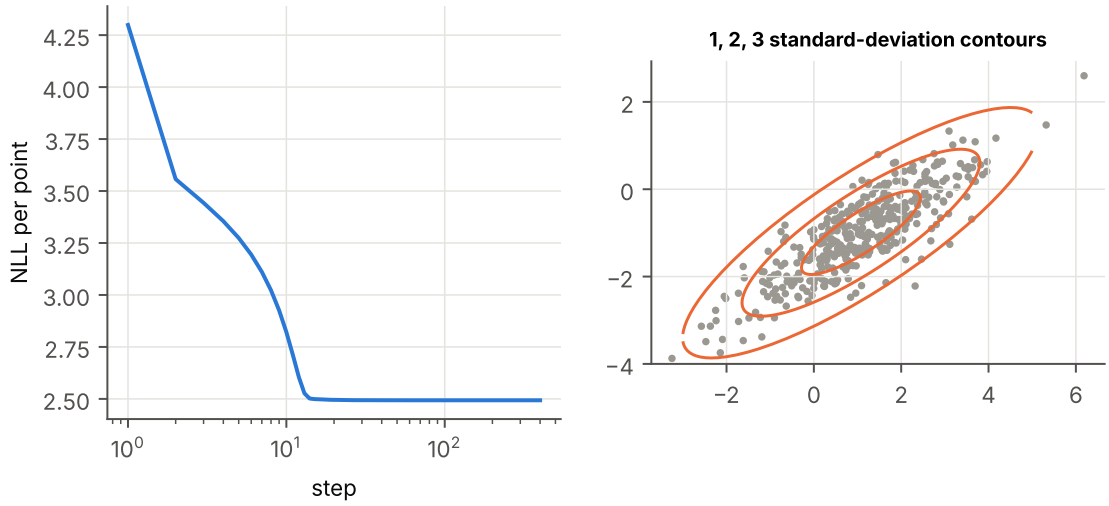


Figure 3.1: Left: negative log-likelihood per point during gradient descent on $(\boldsymbol{\mu}, L)$. Right: the data and the fitted Gaussian's 1-, 2- and 3-standard-deviation contours.

Try it yourself.

1. Change `fit_gaussian_gd` to optimise the precision matrix $\Lambda = LL^\top$ instead of Σ. Derive $\partial E / \partial \Lambda$ first (textbook Section 7.2).
2. Write `kl_gaussians(mu1, S1, mu2, S2)` from textbook Theorem 7.3 and check that it is 0 for identical Gaussians and agrees with `kl_diag_standard`.

CHAPTER 4

Backpropagation: Layers and an MLP

Textbook: Chapter 8. **Assignment:** A4.

4.1 Layers with a forward and a backward method

Each layer stores what its backward pass will need during the forward pass, and its `backward` method turns $\bar{Y}$ (the gradient with respect to its output) into $\bar{X}$, computing parameter gradients on the way. These are textbook Theorems 8.1 and 8.2 line by line.

```
class Linear:
    """Y = X W + b.   backward:  dW = X^T dY,  db = sum_rows(dY),  dX = dY W^T."""

    def __init__(self, d_in, d_out, rng, scale=None):
        scale = np.sqrt(1.0 / d_in) if scale is None else scale
        self.W = rng.standard_normal((d_in, d_out)) * scale
        self.b = np.zeros(d_out)

    def forward(self, X):
        self.X = X                                  # keep the input for backward
        return X @ self.W + self.b

    def backward(self, dY):
        self.dW = self.X.T @ dY
        self.db = dY.sum(axis=0)
        return dY @ self.W.T

    def params(self):
        return [self.W, self.b]

    def grads(self):
        return [self.dW, self.db]


class Tanh:
    """H = tanh(Z).   backward:  dZ = dH * (1 - H^2)."""

    def forward(self, Z):
        self.H = np.tanh(Z)
        return self.H

    def backward(self, dH):
        return dH * (1.0 - self.H ** 2)

    def params(self):
        return []

```

```
    def grads(self):
        return []


class ReLU:
    """H = max(Z, 0).   backward:  dZ = dH * [Z > 0]."""

    def forward(self, Z):
        self.mask = Z > 0
        return Z * self.mask

    def backward(self, dH):
        return dH * self.mask

    def params(self):
        return []

    def grads(self):
        return []


class SoftmaxCrossEntropy:
    """Mean cross-entropy of softmax(A) with integer labels.   backward: (S - T) / N."""

    def forward(self, A, y):
        N = A.shape[0]
        logp = A - logsumexp(A, axis=1, keepdims=True)    # stable log-softmax
        self.S, self.y = np.exp(logp), y
        return float(-np.mean(logp[np.arange(N), y]))

    def backward(self):
        N = self.S.shape[0]
        G = self.S.copy()
        G[np.arange(N), self.y] -= 1.0                    # S - T for one-hot T
        return G / N
```

demos/d04_layers.py

```
import numpy as np
from mcalc import Linear, Tanh, ReLU, SoftmaxCrossEntropy, num_grad, rel_error

rng = np.random.default_rng(0)
X = rng.standard_normal((5, 4))
G = rng.standard_normal((5, 3))          # a random upstream gradient dL/dY

# For a test, use the scalar L = sum(G * layer(X)); then dL/dY = G exactly.
lin = Linear(4, 3, rng)
lin.forward(X); dX = lin.backward(G)
def L_of(W=None, b=None, Xin=None):
    W = lin.W if W is None else W; b = lin.b if b is None else b; Xin = X if Xin is None
    ↪ else Xin
    return np.sum(G * (Xin @ W + b))
print("Linear  dW %.1e  db %.1e  dX %.1e" % (
    rel_error(lin.dW, num_grad(lambda W: L_of(W=W), lin.W)),
```

```
    rel_error(lin.db, num_grad(lambda b: L_of(b=b), lin.b)),
    rel_error(dX, num_grad(lambda Z: L_of(Xin=Z), X))))

for Act in (Tanh, ReLU):
    act = Act(); Z = rng.standard_normal((5, 3))
    act.forward(Z); dZ = act.backward(G)
    print("%-7s dZ %.1e" % (Act.__name__, rel_error(dZ, num_grad(lambda Z: np.sum(G *
    ↪ Act().forward(Z)), Z))))

A = rng.standard_normal((5, 3)); y = np.array([0, 2, 1, 1, 0])
ce = SoftmaxCrossEntropy(); ce.forward(A, y)
print("SoftmaxCrossEntropy dA %.1e" % rel_error(ce.backward(), num_grad(lambda A:
    ↪ SoftmaxCrossEntropy().forward(A, y), A)))
```

OUTPUT

```
Linear  dW 1.7e-10  db 1.2e-10  dX 1.8e-10
Tanh    dZ 1.4e-10
ReLU    dZ 8.2e-11
SoftmaxCrossEntropy dA 8.3e-10
```

To test a layer in isolation we use the scalar $L = \sum_{ij} G_{ij} Y_{ij}$ for a random G: then $\bar{Y} = G$ exactly, and every gradient the layer returns can be compared with finite differences of L. Every check passes at the 10^{-10} level.

4.2 A multilayer perceptron

```
class MLP:
    """A multilayer perceptron with a hand-written backward pass."""

    def __init__(self, sizes, activation="tanh", seed=0):
        rng = np.random.default_rng(seed)
        act = {"tanh": Tanh, "relu": ReLU}[activation]
        self.layers = []
        for i, (d_in, d_out) in enumerate(zip(sizes[:-1], sizes[1:])):
            self.layers.append(Linear(d_in, d_out, rng))
            if i < len(sizes) - 2:                          # no activation after the
    ↪ last layer
                self.layers.append(act())
        self.loss_fn = SoftmaxCrossEntropy()

    def forward(self, X):
        for layer in self.layers:
            X = layer.forward(X)
        return X                                            # logits

    def loss(self, X, y):
        return self.loss_fn.forward(self.forward(X), y)

    def backward(self):
        g = self.loss_fn.backward()
        for layer in reversed(self.layers):
            g = layer.backward(g)
```

```
        return g                                            # gradient w.r.t. the input

    def params(self):
        return [p for layer in self.layers for p in layer.params()]

    def grads(self):
        return [g for layer in self.layers for g in layer.grads()]


def make_spirals(n_per_class=100, n_classes=3, noise=0.2, seed=0):
    """The classic 2-D spiral classification problem."""
    rng = np.random.default_rng(seed)
    X, y = [], []
    for k in range(n_classes):
        r = np.linspace(0.0, 1.0, n_per_class)
        theta = np.linspace(k * 4.0, (k + 1) * 4.0, n_per_class) +
    ↪ rng.standard_normal(n_per_class) * noise
        X.append(np.column_stack([r * np.sin(theta), r * np.cos(theta)]))
        y.append(np.full(n_per_class, k))
    return np.vstack(X), np.concatenate(y)


def train(model, X, y, lr=0.5, epochs=2000, momentum=0.9):
    """Full-batch gradient descent with momentum. Returns the loss history."""
    velocity = [np.zeros_like(p) for p in model.params()]
    history = []
    for _ in range(epochs):
        history.append(model.loss(X, y))
        model.backward()
        for p, g, v in zip(model.params(), model.grads(), velocity):
            v *= momentum
            v -= lr * g
            p += v                                          # in-place: updates the layer
    return np.array(history)
```

`MLP.backward` is the whole of backpropagation: one loop over the layers in reverse. Note that the update `p += v` changes the parameter arrays *in place*, so the layers see the new values without being told.

demos/d04_mlp.py

```
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, savefig
from mcalc import Linear, ReLU, MLP, make_spirals, train, num_grad, rel_error

np.set_printoptions(precision=4, suppress=True)
# Worked Example 8.1: the two-layer ReLU network by hand
x = np.array([[1.0, 1.0]]); t = 3.0
l1, relu, l2 = Linear(2, 2, np.random.default_rng(0)), ReLU(), Linear(2, 1,
    ↪ np.random.default_rng(0))
l1.W = np.array([[1.0, 2.0], [1.0, -1.0]]); l1.b = np.zeros(2)
l2.W = np.array([[1.0], [-1.0]]); l2.b = np.zeros(1)
y = l2.forward(relu.forward(l1.forward(x)))
```

```
print("forward: z =", l1.forward(x), " y =", y, " L =", 0.5 * (y - t) ** 2)
dz = relu.backward(l2.backward(y - t))
dx = l1.backward(dz)
print("dW2 =", l2.dW.ravel(), " dW1 =\n", l1.dW, "\ndx =", dx)

# gradient check of a full MLP, every parameter
X, yl = make_spirals(20, 3, seed=1)
net = MLP([2, 8, 8, 3], activation="tanh", seed=0)
net.loss(X, yl); net.backward()
for i, (p, g) in enumerate(zip(net.params(), net.grads())):
    def L(q, p=p):
        old = p.copy(); p[...] = q; val = net.loss(X, yl); p[...] = old; return val
    print("parameter %d %-8s rel. error %.1e" % (i, str(p.shape), rel_error(g,
    ↪ num_grad(L, p.copy()))))

# train on the spiral data set
X, yl = make_spirals(100, 3, seed=0)
net = MLP([2, 64, 64, 3], activation="tanh", seed=0)
hist = train(net, X, yl, lr=0.2, epochs=3000, momentum=0.9)
acc = np.mean(np.argmax(net.forward(X), axis=1) == yl)
print("loss: start %.3f, after 3000 epochs %.4f; training accuracy %.1f%%" % (hist[0],
    ↪ hist[-1], 100 * acc))
fig, ax = plt.subplots(1, 2, figsize=(7.4, 3.2))
ax[0].semilogy(hist, color=C1); ax[0].set_xlabel("epoch");
    ↪ ax[0].set_ylabel("cross-entropy")
g1, g2 = np.meshgrid(np.linspace(-1.1, 1.1, 300), np.linspace(-1.1, 1.1, 300))
pred = np.argmax(net.forward(np.column_stack([g1.ravel(), g2.ravel()])),
    ↪ axis=1).reshape(g1.shape)
from matplotlib.colors import ListedColormap
ax[1].contourf(g1, g2, pred, levels=[-0.5, 0.5, 1.5, 2.5],
    ↪ cmap=ListedColormap(["#d3e3f6", "#fbd9cb", "#c9ecdd"]))
for k, c in zip(range(3), (C1, C2, C3)):
    ax[1].scatter(X[yl == k, 0], X[yl == k, 1], s=7, color=c)
ax[1].set_aspect("equal"); ax[1].set_xticks([]); ax[1].set_yticks([])
savefig("spirals")
```

OUTPUT

```
forward: z = [[2. 1.]]  y = [[1.]]  L = [[2.]]
dW2 = [-4. -2.]  dW1 =
 [[-2.  2.]
 [-2.  2.]]
dx = [[ 2. -4.]]
parameter 0 (2, 8)   rel. error 2.6e-09
parameter 1 (8,)     rel. error 2.7e-08
parameter 2 (8, 8)   rel. error 4.8e-09
parameter 3 (8,)     rel. error 3.6e-08
parameter 4 (8, 3)   rel. error 2.1e-09
parameter 5 (3,)     rel. error 7.2e-08
loss: start 1.116, after 3000 epochs 0.0115; training accuracy 99.3%
```

The first block is textbook Worked Example 8.1 computed by the layer classes: $\bar{W}_2 = (-4, -2)$, $\bar{W}_1 = \left[\begin{smallmatrix} -2 & 2 \\ -2 & 2 \end{smallmatrix}\right]$ and $\bar{x} = (2, -4)$. The second block checks every parameter of a $2 \to 8 \to 8 \to 3$ network against finite differences. Finally a $2 \to 64 \to 64 \to 3$ network is trained on 300 spiral

points: Figure 4.1 shows the loss and the learned decision regions, with 99% training accuracy.

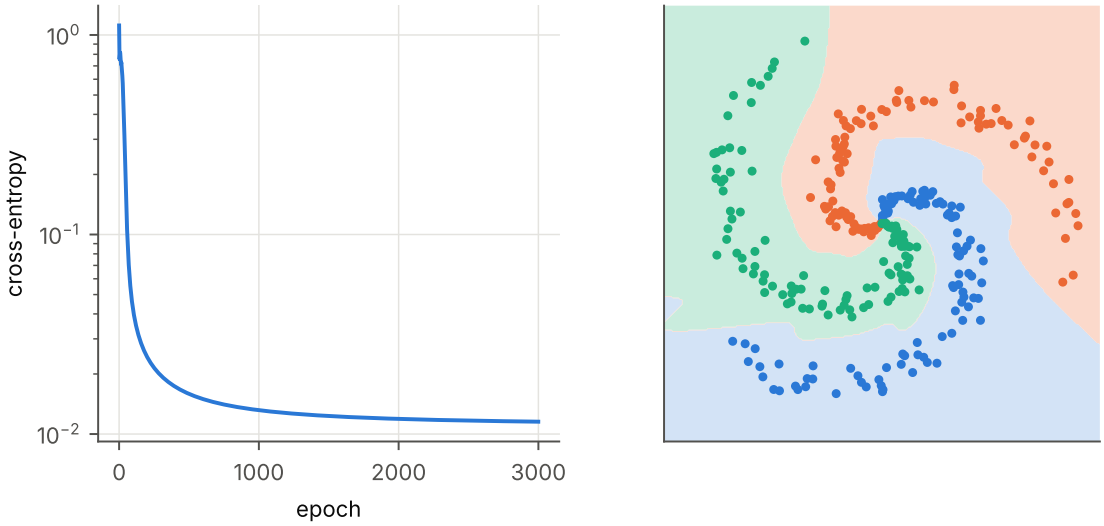


Figure 4.1: Training a two-hidden-layer tanh network with our hand-written backward pass. Left: cross-entropy over 3000 epochs. Right: decision regions on the three-class spiral data.

Try it yourself.

1. Replace `tanh` by `relu` and compare the training curves. Then put a ReLU unit exactly at $z = 0$ and see what the gradient check reports.
2. Add an `MSELoss` layer and a `Sigmoid` layer, with gradient checks, and train the network on a regression problem.

CHAPTER 5

Automatic Differentiation and Numerical Stability

Textbook: Chapter 9. **Assignment:** A5.

5.1 Forward mode: dual numbers

A dual number carries a value and a derivative; each operation updates both by the rules of textbook Section 9.3. The functions `sin`, `exp`, ... accept either an ordinary number or a dual number, so the same code computes a value or a derivative.

```
class Dual:
    """Dual number a + b eps with eps^2 = 0: forward-mode automatic differentiation."""

    def __init__(self, val, der=0.0):
        self.val, self.der = float(val), float(der)

    @staticmethod
    def _lift(x):
        return x if isinstance(x, Dual) else Dual(x, 0.0)

    def __add__(self, o):
        o = Dual._lift(o)
        return Dual(self.val + o.val, self.der + o.der)
    __radd__ = __add__

    def __sub__(self, o):
        o = Dual._lift(o)
        return Dual(self.val - o.val, self.der - o.der)

    def __rsub__(self, o):
        return Dual._lift(o) - self

    def __mul__(self, o):
        o = Dual._lift(o)
        return Dual(self.val * o.val, self.val * o.der + self.der * o.val)
    __rmul__ = __mul__

    def __truediv__(self, o):
        o = Dual._lift(o)
        return Dual(self.val / o.val, (self.der * o.val - self.val * o.der) / o.val ** 2)

    def __rtruediv__(self, o):
        return Dual._lift(o) / self

```

```
    def __neg__(self):
        return Dual(-self.val, -self.der)

    def __pow__(self, p):                    # real exponent p
        return Dual(self.val ** p, p * self.val ** (p - 1) * self.der)

    def __repr__(self):
        return f"Dual({self.val:.6g}, {self.der:.6g})"


def _unary(fn, dfn):
    def op(x):
        if isinstance(x, Dual):
            return Dual(fn(x.val), dfn(x.val) * x.der)
        return fn(x)
    return op


sin = _unary(np.sin, np.cos)
cos = _unary(np.cos, lambda v: -np.sin(v))
exp = _unary(np.exp, np.exp)
log = _unary(np.log, lambda v: 1.0 / v)
tanh = _unary(np.tanh, lambda v: 1.0 - np.tanh(v) ** 2)


def derivative(f, x):
    """f'(x) by evaluating f on the dual number x + 1 eps."""
    return f(Dual(x, 1.0)).der
```

5.2 Reverse mode: a Tensor that records its history

Each operation on a `Tensor` creates a new `Tensor` that remembers its parents and a small `_backward` function — the local rule for passing $\bar{\text{out}}$ to the parents. `backward()` sorts the graph topologically and runs those rules from the output back to the inputs. This is a complete reverse-mode engine in under 200 lines, and every rule in it is one you derived: $\bar{X} = \bar{Y}W^{\mathsf{T}}$ and $\bar{W} = X^{\mathsf{T}}\bar{Y}$ for `@`, $\bar{x} = \bar{y}\,y$ for `exp`, and so on.

```
def _unbroadcast(grad, shape):
    """Sum a gradient over the axes that broadcasting expanded, back to `shape`."""
    while grad.ndim > len(shape):
        grad = grad.sum(axis=0)
    for axis, n in enumerate(shape):
        if n == 1 and grad.shape[axis] != 1:
            grad = grad.sum(axis=axis, keepdims=True)
    return grad


class Tensor:
    """A NumPy array that records operations, for reverse-mode differentiation."""

    def __init__(self, data, parents=(), op=""):
        self.data = np.asarray(data, dtype=float)
```

```
        self.grad = np.zeros_like(self.data)
        self._parents = parents
        self._backward = lambda: None
        self.op = op

    @property
    def shape(self):
        return self.data.shape

    @staticmethod
    def _lift(x):
        return x if isinstance(x, Tensor) else Tensor(x)

    # ---- elementary operations: each defines the forward value and its backward rule
    def __add__(self, other):
        other = Tensor._lift(other)
        out = Tensor(self.data + other.data, (self, other), "+")
        def _backward():
            self.grad += _unbroadcast(out.grad, self.shape)
            other.grad += _unbroadcast(out.grad, other.shape)
        out._backward = _backward
        return out
    __radd__ = __add__

    def __mul__(self, other):
        other = Tensor._lift(other)
        out = Tensor(self.data * other.data, (self, other), "*")
        def _backward():
            self.grad += _unbroadcast(out.grad * other.data, self.shape)
            other.grad += _unbroadcast(out.grad * self.data, other.shape)
        out._backward = _backward
        return out
    __rmul__ = __mul__

    def __neg__(self):
        return self * -1.0

    def __sub__(self, other):
        return self + (-Tensor._lift(other))

    def __rsub__(self, other):
        return Tensor._lift(other) + (-self)

    def __pow__(self, p):                          # real exponent p
        out = Tensor(self.data ** p, (self,), f"**{p}")
        def _backward():
            self.grad += out.grad * p * self.data ** (p - 1)
        out._backward = _backward
        return out

    def __truediv__(self, other):
        return self * Tensor._lift(other) ** -1.0

    def __matmul__(self, other):                   # 2-D matrices
```

```
        other = Tensor._lift(other)
        out = Tensor(self.data @ other.data, (self, other), "@")
        def _backward():
            self.grad += out.grad @ other.data.T          # dX = dY W^T
            other.grad += self.data.T @ out.grad          # dW = X^T dY
        out._backward = _backward
        return out

    @property
    def T(self):
        out = Tensor(self.data.T, (self,), "T")
        def _backward():
            self.grad += out.grad.T
        out._backward = _backward
        return out

    def sum(self, axis=None, keepdims=False):
        out = Tensor(self.data.sum(axis=axis, keepdims=keepdims), (self,), "sum")
        def _backward():
            g = out.grad
            if axis is not None and not keepdims:
                g = np.expand_dims(g, axis)
            self.grad += np.broadcast_to(g, self.shape)
        out._backward = _backward
        return out

    def mean(self, axis=None):
        n = self.data.size if axis is None else self.data.shape[axis]
        return self.sum(axis=axis) * (1.0 / n)

    def _elementwise(self, value, local_grad, op):
        out = Tensor(value, (self,), op)
        def _backward():
            self.grad += out.grad * local_grad
        out._backward = _backward
        return out

    def exp(self):
        e = np.exp(self.data)
        return self._elementwise(e, e, "exp")

    def log(self):
        return self._elementwise(np.log(self.data), 1.0 / self.data, "log")

    def tanh(self):
        t = np.tanh(self.data)
        return self._elementwise(t, 1.0 - t ** 2, "tanh")

    def relu(self):
        return self._elementwise(np.maximum(self.data, 0.0), (self.data >
    ↪ 0).astype(float), "relu")

    def sigmoid(self):
        s = sigmoid(self.data)
```

```
        return self._elementwise(s, s * (1.0 - s), "sigmoid")

    # ---- the reverse sweep
    def backward(self):
        """Fill .grad of every tensor this one depends on (self must be a scalar)."""
        order, seen = [], set()
        def visit(v):                              # depth-first topological sort
            if id(v) not in seen:
                seen.add(id(v))
                for p in v._parents:
                    visit(p)
                order.append(v)
        visit(self)
        self.grad = np.ones_like(self.data)        # seed: dL/dL = 1
        for v in reversed(order):
            v._backward()


def cross_entropy(logits, y):
    """Mean softmax cross-entropy as ONE primitive: stable forward, (S - T)/N
    ↪ backward."""
    N = logits.shape[0]
    logp = logits.data - logsumexp(logits.data, axis=1, keepdims=True)
    out = Tensor(-np.mean(logp[np.arange(N), y]), (logits,), "xent")
    def _backward():
        G = np.exp(logp)
        G[np.arange(N), y] -= 1.0
        logits.grad += out.grad * G / N
    out._backward = _backward
    return out


def zero_grad(params):
    for p in params:
        p.grad = np.zeros_like(p.data)
```

Three details deserve attention:

- **Gradients accumulate** (`+=`): a tensor used twice receives two contributions, exactly as x_1 did in textbook Worked Example 9.3.
- **Broadcasting has a backward pass.** If a bias of shape `(3,)` is added to every row of a `(5, 3)` matrix, its gradient is the sum of the five row gradients. `_unbroadcast` performs that sum.
- **`cross_entropy` is one primitive**, not a chain of `exp`, `sum` and `log`: its forward pass is the stable log-softmax and its backward pass is the clean $(S - T)/N$.

demos/d05_autodiff.py

```python
import numpy as np
from mcalc import Dual, derivative, sin, exp, Tensor, cross_entropy, zero_grad, MLP, make_spirals, num_grad, rel_error

np.set_printoptions(precision=4, suppress=True)
# forward mode: dual numbers (Worked Example 9.2, Exercise 9.2)
print("f(x) = x^3 + 2x at 2:", (lambda x: x ** 3 + 2 * x)(Dual(2.0, 1.0)))
print("f'(1) for (x^2+1)(3x-1):", derivative(lambda x: (x ** 2 + 1) * (3 * x - 1), 1.0))
g = lambda x: exp(sin(x)) * x
print("d/dx x exp(sin x) at 1: dual %.12f, central diff %.12f"
      % (derivative(g, 1.0), (g(1 + 1e-6) - g(1 - 1e-6)) / 2e-6))

# reverse mode: the sweep of Worked Example 9.3
x1, x2 = Tensor(1.0), Tensor(2.0)
f = (x1 * x2 + x1) ** 2
f.backward()
print("f = %.0f, df/dx1 = %.0f, df/dx2 = %.0f" % (f.data, x1.grad, x2.grad))

# a composite function of matrices, checked against finite differences
rng = np.random.default_rng(0)
A0, B0 = rng.standard_normal((3, 4)), rng.standard_normal((4, 2))
def F(A, B):
    return ((A @ B).tanh() * 2.0 + 1.0).exp().sum() / 10.0
A, B = Tensor(A0), Tensor(B0)
out = F(A, B); out.backward()
fA = lambda a: float(F(Tensor(a), Tensor(B0)).data)
fB = lambda b: float(F(Tensor(A0), Tensor(b)).data)
print("composite: dA error %.1e, dB error %.1e" % (rel_error(A.grad, num_grad(fA, A0)), rel_error(B.grad, num_grad(fB, B0))))

# broadcasting: a bias added to every row receives the sum of the row gradients
W, b = Tensor(rng.standard_normal((2, 3))), Tensor(np.zeros(3))
Xb = Tensor(rng.standard_normal((5, 2)))
((Xb @ W + b) ** 2).sum().backward()
print("bias gradient equals column sums:", np.allclose(b.grad, (2 * (Xb.data @ W.data + b.data)).sum(0)))

# the same MLP gradient two ways: hand-written backprop vs our autodiff engine
X, y = make_spirals(20, 3, seed=1)
net = MLP([2, 8, 3], activation="tanh", seed=0)
net.loss(X, y); net.backward()
W1, b1, W2, b2 = [Tensor(p.copy()) for p in net.params()]
loss = cross_entropy((Tensor(X) @ W1 + b1).tanh() @ W2 + b2, y)
loss.backward()
diffs = [np.abs(t.grad - g).max() for t, g in zip((W1, b1, W2, b2), net.grads())]
print("hand backprop vs autodiff, max differences:", " ".join("%.1e" % d for d in diffs))
```

OUTPUT

```
f(x) = x^3 + 2x at 2: Dual(12, 14)
f'(1) for (x^2+1)(3x-1): 10.0
d/dx x exp(sin x) at 1: dual 3.573157592209, central diff 3.573157592252
f = 9, df/dx1 = 18, df/dx2 = 6
composite: dA error 2.6e-10, dB error 1.8e-10
```

```
bias gradient equals column sums: True
hand backprop vs autodiff, max differences: 0.0e+00 0.0e+00 0.0e+00 0.0e+00
```

Dual numbers reproduce textbook Worked Example 9.2 and Exercise 9.2, and agree with central differences to 10^{-11} on $x\,e^{\sin x}$. The Tensor engine reproduces the reverse sweep of Worked Example 9.3 and passes a gradient check on a composite matrix function. Finally, the gradients of a whole MLP computed by our hand-written backward pass (Part 4) and by the autodiff engine are *identical* — not merely close: both perform the same floating-point operations in the same order.

5.3 Numerical stability

demos/d05_stability.py

```python
import warnings
import numpy as np
from mcalc import logsumexp, softmax, softplus, log_sigmoid

np.set_printoptions(precision=4)
a = np.array([1000.0, 1001.0, 1002.0])                                  # Worked Example 9.4
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    naive = np.exp(a) / np.sum(np.exp(a))
    print("naive softmax :", naive)
    print("naive lse     :", np.log(np.sum(np.exp(a))))
    print("naive softplus(1000):", np.log(1 + np.exp(1000.0)))
    print("naive log sigmoid(-800):", np.log(1 / (1 + np.exp(800.0))))
print("stable softmax:", softmax(a))
print("stable lse    : %.4f" % logsumexp(a))
print("stable softplus(1000) = %.1f, softplus(-1000) = %.1f" % (softplus(1000.0),
    ↪ softplus(-1000.0)))
print("stable log sigmoid(-800) = %.1f" % log_sigmoid(-800.0))

# binary cross-entropy from probabilities vs from logits
t, logit = 1.0, -40.0
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    p = 1 / (1 + np.exp(-logit))
    print("BCE via probability: %.4f   (p = %.3e)" % (-(t * np.log(p) + (1 - t) *
    ↪ np.log(1 - p)), p))
print("BCE via logits     : %.4f" % (softplus(logit) - t * logit))

# log-determinant and solves: never invert
rng = np.random.default_rng(0)
B = rng.standard_normal((300, 300)); S = B @ B.T + 1e-3 * np.eye(300)
sign, ld = np.linalg.slogdet(S)
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    print("log(det(S)) =", np.log(np.linalg.det(S)), "   slogdet =", round(ld, 4))
# solve versus inverse on the Hilbert matrix h_ij = 1/(i+j-1), kappa ~ 1.6e13
n = 10
Hm = np.array([[1.0 / (i + j + 1) for j in range(n)] for i in range(n)])
```

```
x_true = np.ones(n); rhs = Hm @ x_true
y_solve = np.linalg.solve(Hm, rhs); y_inv = np.linalg.inv(Hm) @ rhs
print("Hilbert n=10: residual |H y - b|  solve %.1e, inv %.1e" % (np.linalg.norm(Hm @ y_solve - rhs), np.linalg.norm(Hm @ y_inv - rhs)))
print("              error    |y - x|    solve %.1e, inv %.1e" % (np.linalg.norm(y_solve - x_true), np.linalg.norm(y_inv - x_true)))
```

OUTPUT

```
naive softmax : [nan nan nan]
naive lse     : inf
naive softplus(1000): inf
naive log sigmoid(-800): -inf
stable softmax: [0.09   0.2447 0.6652]
stable lse    : 1002.4076
stable softplus(1000) = 1000.0, softplus(-1000) = 0.0
stable log sigmoid(-800) = -800.0
BCE via probability: 40.0000   (p = 4.248e-18)
BCE via logits     : 40.0000
log(det(S)) = inf    slogdet = 1404.213
Hilbert n=10: residual |H y - b|  solve 7.4e-16, inv 1.6e-04
              error    |y - x|    solve 4.0e-05, inv 7.7e-03
```

Each naive formula fails in the way textbook Section 9.5 predicts — `nan`, `inf` or $-\infty$ — while the stable forms give the correct values. Two less obvious lessons:

- The binary cross-entropy of a confident wrong prediction (logit -40) is 40. Computed from the probability it happens to come out right here, but at logit -800 the probability underflows to 0 and the loss becomes infinite; computed from the logit it is always exact.
- On the Hilbert matrix ($\kappa \approx 10^{13}$), `solve` leaves a residual of 10^{-16}, the explicit inverse one of 10^{-4} — twelve digits lost by one unnecessary `inv`.

5.4 Optional: checking against JAX

JAX is a NumPy-like library whose `jax.grad` differentiates Python functions automatically. It is a good *independent* check of hand-derived gradients, because its answer is exact up to rounding rather than approximate like finite differences. Two settings matter: enable 64-bit floats (JAX defaults to 32-bit), and remember that JAX arrays are immutable. The demo is skipped automatically if JAX is not installed.

demos/d05_jax.py

```
# Optional: compare our hand-derived gradients with JAX's automatic differentiation.
import numpy as np
from mcalc import CATALOGUE, MLP, make_spirals, softmax_jacobian

try:
    import jax
    import jax.numpy as jnp
    jax.config.update("jax_enable_x64", True)        # JAX defaults to float32
except ImportError:
    print("JAX is not installed -- this optional demo is skipped (pip install jax).")
```

```
    raise SystemExit

print("JAX version", jax.__version__)
rng = np.random.default_rng(0)
n = 4
A = rng.standard_normal((n, n)); X = rng.standard_normal((n, n)) + 2 * np.eye(n)
checks = {
    "ln|det X|":   (lambda X: jnp.linalg.slogdet(X)[1], {}),
    "tr(X^-1 A)":  (lambda X: jnp.trace(jnp.linalg.solve(X, A)), {"A": A}),
    "tr(X^T A X)": (lambda X: jnp.trace(X.T @ A @ X), {"A": A}),
}
for name, (fj, P) in checks.items():
    ours = CATALOGUE[name][1](X, P)
    print("%-12s max |ours - jax.grad| = %.1e" % (name, np.abs(ours -
    ↪ np.asarray(jax.grad(fj)(X))).max()))

# Jacobians: forward mode (jacfwd) and reverse mode (jacrev) give the same matrix
a = np.array([0.0, np.log(2.0), 0.0])
sm = lambda a: jax.nn.softmax(a)
print("softmax Jacobian: |ours - jacfwd| = %.1e, |ours - jacrev| = %.1e"
      % (np.abs(softmax_jacobian(a) - jax.jacfwd(sm)(a)).max(),
    ↪ np.abs(softmax_jacobian(a) - jax.jacrev(sm)(a)).max()))

# a whole MLP: our hand-written backward pass vs jax.grad
Xd, y = make_spirals(20, 3, seed=1)
net = MLP([2, 8, 8, 3], activation="tanh", seed=0)
net.loss(Xd, y); net.backward()
def loss(params):
    h = Xd
    for i in range(0, len(params) - 2, 2):
        h = jnp.tanh(h @ params[i] + params[i + 1])
    logits = h @ params[-2] + params[-1]
    logp = logits - jax.scipy.special.logsumexp(logits, axis=1, keepdims=True)
    return -jnp.mean(logp[jnp.arange(len(y)), y])
jg = jax.grad(loss)([jnp.asarray(p) for p in net.params()])
print("MLP: max |ours - jax.grad| per parameter:", " ".join("%.1e" % np.abs(g -
    ↪ np.asarray(j)).max() for g, j in zip(net.grads(), jg)))
```

OUTPUT

```
JAX version 0.11.2
ln|det X|    max |ours - jax.grad| = 4.4e-16
tr(X^-1 A)   max |ours - jax.grad| = 2.2e-15
tr(X^T A X)  max |ours - jax.grad| = 6.7e-16
softmax Jacobian: |ours - jacfwd| = 0.0e+00, |ours - jacrev| = 0.0e+00
MLP: max |ours - jax.grad| per parameter: 1.4e-17 5.5e-17 4.2e-17 3.9e-17 2.8e-17 5.4e-17
```

Our matrix derivatives agree with `jax.grad` to 10^{-15}, our softmax Jacobian with both `jax.jacfwd` (forward mode) and `jax.jacrev` (reverse mode) exactly, and our hand-written MLP backward pass with `jax.grad` of the same network to 10^{-17}. Nothing in the assignments needs JAX; it is here so that you know your derivations are what a production autodiff system computes.

Try it yourself.

1. Add `__getitem__` (indexing) to `Tensor`. What is the backward rule of `x[i]`?
2. Use `Dual` to compute a Jacobian–vector product Jv of a function $\mathbb{R}^3 \to \mathbb{R}^2$, and compare with `num_jacobian(f, x) @ v`.

CHAPTER 6

Attention, Layer Normalisation and Flows

Textbook: Chapters 10–12. **Assignment:** A6.

6.1 Self-attention

```python
def softmax_rows_backward(A, dA):
    """Backward pass of a row-wise softmax: dS = A * (dA - rowsum(dA * A))."""
    return A * (dA - np.sum(dA * A, axis=1, keepdims=True))


def attention_forward(X, Wq, Wk, Wv, causal=False):
    """One self-attention head. Returns Y and a cache for the backward pass."""
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    d_k = Wq.shape[1]
    S = Q @ K.T / np.sqrt(d_k)
    if causal:                                      # token i may only attend to j <= i
        S = np.where(np.tril(np.ones_like(S, dtype=bool)), S, -np.inf)
    A = softmax(S, axis=1)
    Y = A @ V
    return Y, (X, Wq, Wk, Wv, Q, K, V, A, d_k)


def attention_backward(dY, cache):
    """Gradients of self-attention: returns dX, dWq, dWk, dWv."""
    X, Wq, Wk, Wv, Q, K, V, A, d_k = cache
    dV = A.T @ dY
    dA = dY @ V.T
    dS = softmax_rows_backward(A, dA)
    dQ = dS @ K / np.sqrt(d_k)
    dK = dS.T @ Q / np.sqrt(d_k)
    dWq, dWk, dWv = X.T @ dQ, X.T @ dK, X.T @ dV
    dX = dQ @ Wq.T + dK @ Wk.T + dV @ Wv.T          # X is used three times: add
    return dX, dWq, dWk, dWv
```

The backward function is textbook Theorem 12.2 in eight lines. The causal mask sets disallowed scores to $-\infty$, so their attention weights are exactly 0 and, by the softmax backward rule, so are their gradients — no special case is needed in the backward pass.

demos/d06_attention.py

```python
import numpy as np
from mcalc import attention_forward, attention_backward, softmax_rows_backward, softmax, num_grad, rel_error

np.set_printoptions(precision=4, suppress=True)
# Worked Example 12.1: two tokens, d = 1, all weights 1, loss L = y_1
X = np.array([[0.0], [1.0]]); I1 = np.eye(1)
Y, cache = attention_forward(X, I1, I1, I1)
print("Y =", Y.ravel(), "  A =\n", cache[7])
dX, dWq, dWk, dWv = attention_backward(np.array([[1.0], [0.0]]), cache)
print("dX =", dX.ravel(), " (expected 0.75, 0.5)")

# softmax-row backward vs finite differences
rng = np.random.default_rng(0)
S = rng.standard_normal((4, 5)); G = rng.standard_normal((4, 5))
print("softmax rows: %.1e" % rel_error(softmax_rows_backward(softmax(S, axis=1), G),
                                       num_grad(lambda S: np.sum(G * softmax(S, axis=1)), S)))

# full gradient check of one head, with and without a causal mask
N, D, dk = 6, 5, 3
X = rng.standard_normal((N, D)); Wq, Wk, Wv = (rng.standard_normal((D, dk)) for _ in range(3))
G = rng.standard_normal((N, dk))
for causal in (False, True):
    def L(X=X, Wq=Wq, Wk=Wk, Wv=Wv):
        return np.sum(G * attention_forward(X, Wq, Wk, Wv, causal)[0])
    _, cache = attention_forward(X, Wq, Wk, Wv, causal)
    dX, dWq, dWk, dWv = attention_backward(G, cache)
    print("causal=%-5s dX %.1e  dWq %.1e  dWk %.1e  dWv %.1e" % (causal,
          rel_error(dX, num_grad(lambda Z: L(X=Z), X)), rel_error(dWq, num_grad(lambda W: L(Wq=W), Wq)),
          rel_error(dWk, num_grad(lambda W: L(Wk=W), Wk)), rel_error(dWv, num_grad(lambda W: L(Wv=W), Wv))))

# why divide by sqrt(d_k): the size of the softmax Jacobian with and without scaling
for d in (4, 64, 512):
    q, K = rng.standard_normal(d), rng.standard_normal((16, d))
    for scaled in (False, True):
        s = softmax(K @ q / (np.sqrt(d) if scaled else 1.0))
        J = np.diag(s) - np.outer(s, s)
        print("d=%3d scaled=%-5s max attention %.3f, |J|_F = %.3f" % (d, scaled, s.max(), np.linalg.norm(J)))
```

OUTPUT

```
Y = [0.5    0.7311]   A =
 [[0.5    0.5   ]
 [0.2689 0.7311]]
dX = [0.75 0.5 ]  (expected 0.75, 0.5)
softmax rows: 3.3e-10
causal=False dX 9.1e-10  dWq 1.9e-09  dWk 1.5e-09  dWv 2.8e-10
causal=True  dX 1.7e-10  dWq 2.2e-10  dWk 2.5e-10  dWv 1.4e-10
d=  4 scaled=False max attention 0.375, |J|_F = 0.371
```

```
d=  4 scaled=True  max attention 0.205, |J|_F = 0.303
d= 64 scaled=False max attention 0.500, |J|_F = 0.499
d= 64 scaled=True  max attention 0.195, |J|_F = 0.297
d=512 scaled=False max attention 1.000, |J|_F = 0.000
d=512 scaled=True  max attention 0.405, |J|_F = 0.311
```

The first block is textbook Worked Example 12.1: $\bar{X} = (0.75, 0.5)$. The full gradient check covers every input and weight of one head, with and without the causal mask. The last block shows why scores are divided by $\sqrt{d_k}$ (textbook Exercise 12.6): with $d = 512$ and no scaling, the attention row is one-hot and its Jacobian has norm 0 — no gradient would reach the queries and keys. With scaling the Jacobian keeps a healthy norm for every d.

6.2 Layer normalisation

```python
def layernorm_forward(X, gamma, beta, eps=1e-5):
    """Normalise each row of X, then scale and shift."""
    mu = X.mean(axis=1, keepdims=True)
    var = X.var(axis=1, keepdims=True)
    sig = np.sqrt(var + eps)
    Xh = (X - mu) / sig
    return gamma * Xh + beta, (Xh, sig, gamma)


def layernorm_backward(dY, cache):
    """dX = (dXh - mean(dXh) - Xh mean(dXh * Xh)) / sigma, row by row."""
    Xh, sig, gamma = cache
    dXh = dY * gamma
    dX = (dXh - dXh.mean(axis=1, keepdims=True)
          - Xh * (dXh * Xh).mean(axis=1, keepdims=True)) / sig
    return dX, (dY * Xh).sum(axis=0), dY.sum(axis=0)
```

demos/d06_layernorm.py

```python
import numpy as np
from mcalc import layernorm_forward, layernorm_backward, num_grad, num_jacobian,
    ↪ rel_error

np.set_printoptions(precision=4, suppress=True)
x = np.array([[1.0, 2.0, 3.0]])                                  # Worked Example
    ↪ 12.2
J = num_jacobian(lambda v: layernorm_forward(v.reshape(1, 3), np.ones(3), np.zeros(3),
    ↪ eps=0.0)[0], x)
print("numerical Jacobian of the standardisation:\n", J)
print("J 1 =", J @ np.ones(3), "  J x =", J @ x.ravel())

rng = np.random.default_rng(0)
X = rng.standard_normal((4, 6)); gamma = rng.standard_normal(6); beta =
    ↪ rng.standard_normal(6)
G = rng.standard_normal((4, 6))
_, cache = layernorm_forward(X, gamma, beta)
dX, dgamma, dbeta = layernorm_backward(G, cache)
L = lambda X=X, g=gamma, b=beta: np.sum(G * layernorm_forward(X, g, b)[0])
```

```
print("layer norm: dX %.1e, dgamma %.1e, dbeta %.1e" % (
    rel_error(dX, num_grad(lambda Z: L(X=Z), X)),
    rel_error(dgamma, num_grad(lambda g: L(g=g), gamma)),
    rel_error(dbeta, num_grad(lambda b: L(b=b), beta))))
```

OUTPUT

```
numerical Jacobian of the standardisation:
 [[ 0.2041 -0.4082  0.2041]
 [-0.4082  0.8165 -0.4082]
 [ 0.2041 -0.4082  0.2041]]
J 1 = [ 0. -0. -0.]   J x = [ 0. -0. -0.]
layer norm: dX 5.4e-10, dgamma 2.3e-10, dbeta 2.0e-10
```

The numerical Jacobian of the standardisation at $\boldsymbol{x} = (1, 2, 3)$ is exactly the matrix of textbook Worked Example 12.2, and it annihilates both $\mathbf{1}$ and $\boldsymbol{x}$ (shift and scale invariance). The backward pass passes the gradient check for X, γ and β.

6.3 Normalizing flows, Hutchinson and PCA

```
def coupling_forward(x, Ws, bs, Wt, bt):
    """Affine coupling layer on the rows of x (N x D, D even).

    z_A = x_A,  z_B = x_B * exp(s) + t,  s = tanh(x_A Ws + bs),  t = x_A Wt + bt.
    Returns z and log|det J| = sum(s) for every row.
    """
    d = x.shape[1] // 2
    xa, xb = x[:, :d], x[:, d:]
    s = np.tanh(xa @ Ws + bs)
    t = xa @ Wt + bt
    return np.hstack([xa, xb * np.exp(s) + t]), s.sum(axis=1)


def coupling_inverse(z, Ws, bs, Wt, bt):
    d = z.shape[1] // 2
    za, zb = z[:, :d], z[:, d:]
    s = np.tanh(za @ Ws + bs)
    t = za @ Wt + bt
    return np.hstack([za, (zb - t) * np.exp(-s)])


def linear_flow_nll(X, A, b):
    """Mean NLL of x = A z + b, z ~ N(0, I), with gradients w.r.t. A and b.

    dA = A^{-T} (I - mean(z z^T)),   db = -A^{-T} mean(z).
    """
    N, D = X.shape
    Z = np.linalg.solve(A, (X - b).T).T            # rows z_n = A^{-1}(x_n - b)
    value = 0.5 * D * np.log(2 * np.pi) + 0.5 * np.mean(np.sum(Z ** 2, axis=1)) \
        + np.linalg.slogdet(A)[1]
    AinvT = np.linalg.inv(A).T
    dA = AinvT @ (np.eye(D) - Z.T @ Z / N)
    db = -AinvT @ Z.mean(axis=0)
```

```
    return float(value), dA, db


def hutchinson_samples(matvec, n, n_samples, rng, kind="rademacher"):
    """Individual Hutchinson samples eps^T A eps (their mean estimates tr A)."""
    out = np.empty(n_samples)
    for i in range(n_samples):
        eps = rng.choice([-1.0, 1.0], size=n) if kind == "rademacher" else
    ↪ rng.standard_normal(n)
        out[i] = eps @ matvec(eps)
    return out


def rayleigh_quotient(S, x):
    """R(x) = x^T S x / x^T x and its gradient 2 (S x - R x) / x^T x."""
    xx = x @ x
    R = x @ S @ x / xx
    return float(R), 2.0 * (S @ x - R * x) / xx
```

demos/d06_flows.py

```
import numpy as np
import matplotlib.pyplot as plt
from plotstyle import C1, C2, C3, INK2, savefig
from mcalc import (linear_flow_nll, coupling_forward, coupling_inverse, num_jacobian,
    ↪ num_grad,
                   rel_error, hutchinson_samples, rayleigh_quotient)

np.set_printoptions(precision=4, suppress=True)
A = np.array([[2.0, 0.0], [1.0, 1.0]])                              # Worked Example
    ↪ 11.1
val, _, _ = linear_flow_nll(np.array([[2.0, 3.0]]), A, np.zeros(2))
print("ln p(x) = %.4f   (-ln 2pi - 2.5 - ln 2 = %.4f)" % (-val, -np.log(2 * np.pi) - 2.5
    ↪ - np.log(2)))

rng = np.random.default_rng(0)
C = np.array([[2.0, 1.2], [1.2, 1.0]])
X = rng.multivariate_normal([1.0, -1.0], C, size=500)
A0, b0 = np.eye(2) + 0.1 * rng.standard_normal((2, 2)), np.zeros(2)
_, dA, db = linear_flow_nll(X, A0, b0)
print("flow gradient check: dA %.1e, db %.1e" % (rel_error(dA, num_grad(lambda M:
    ↪ linear_flow_nll(X, M, b0)[0], A0)),
                                                 rel_error(db, num_grad(lambda v:
    ↪ linear_flow_nll(X, A0, v)[0], b0))))
A, b = np.eye(2), np.zeros(2)
for _ in range(2000):
    _, dA, db = linear_flow_nll(X, A, b)
    A, b = A - 0.05 * dA, b - 0.05 * db
mu_ml = X.mean(0); S_ml = (X - mu_ml).T @ (X - mu_ml) / len(X)
print("trained flow: A A^T =\n", A @ A.T, "\nsample covariance =\n", S_ml, "\nb =", b, "
    ↪ mean =", mu_ml)

# a coupling layer: log|det J| from the formula vs a numerical Jacobian
D = 4
```

```
Ws, Wt = rng.standard_normal((2, 2)), rng.standard_normal((2, 2))
bs, bt = rng.standard_normal(2), rng.standard_normal(2)
x = rng.standard_normal((1, D))
z, logdet = coupling_forward(x, Ws, bs, Wt, bt)
J = num_jacobian(lambda v: coupling_forward(v.reshape(1, D), Ws, bs, Wt, bt)[0], x)
print("coupling: formula log|det J| = %.6f, numerical = %.6f" % (logdet[0],
    ↪ np.linalg.slogdet(J)[1]))
print("upper-right block of J (must be 0):\n", J[:2, 2:])
print("inverse recovers x:", np.allclose(coupling_inverse(z, Ws, bs, Wt, bt), x))

# Hutchinson's trace estimator: running mean with Rademacher and Gaussian vectors
n = 50
B = rng.standard_normal((n, n)); M = B @ B.T / n
k = np.arange(1, 2001)
fig, ax = plt.subplots(figsize=(6.0, 3.0))
for kind, c in (("rademacher", C1), ("gaussian", C2)):
    s = hutchinson_samples(lambda v: M @ v, n, 2000, np.random.default_rng(1), kind)
    ax.loglog(k, np.abs(np.cumsum(s) / k - np.trace(M)) / np.trace(M), color=c,
    ↪ label=kind, lw=1.2)
    print("%-10s sample std %.2f, estimate after 2000 samples %.3f (true trace %.3f)" %
    ↪ (kind, s.std(), s.mean(), np.trace(M)))
ax.loglog(k, 2 / np.sqrt(k), color=INK2, ls=":", lw=0.9)
ax.text(300, 0.15, "$\\propto 1/\\sqrt{k}$", color=INK2)
ax.set_xlabel("number of samples $k$"); ax.set_ylabel("relative error"); ax.legend();
    ↪ savefig("hutchinson")

# PCA by gradient ascent on the Rayleigh quotient (Chapter 10)
S = np.array([[2.0, 1.0], [1.0, 2.0]])
x = np.array([1.0, 0.0])
for _ in range(50):
    R, g = rayleigh_quotient(S, x)
    x = x + 0.5 * g
    x = x / np.linalg.norm(x)
print("Rayleigh ascent: R = %.6f, x = %s   (top eigenpair: 3, [0.7071 0.7071])" %
    ↪ (rayleigh_quotient(S, x)[0], x))
```

OUTPUT

```
ln p(x) = -5.0310   (-ln 2pi - 2.5 - ln 2 = -5.0310)
flow gradient check: dA 4.9e-10, db 9.5e-11
trained flow: A A^T =
 [[1.9402 1.164 ]
 [1.164  0.9601]]
sample covariance =
 [[1.9402 1.164 ]
 [1.164  0.9601]]
b = [ 1.1243 -0.9216]  mean = [ 1.1243 -0.9216]
coupling: formula log|det J| = -1.712236, numerical = -1.712236
upper-right block of J (must be 0):
 [[0. 0.]
 [0. 0.]]
inverse recovers x: True
rademacher sample std 10.02, estimate after 2000 samples 51.293 (true trace 51.334)
gaussian   sample std 14.69, estimate after 2000 samples 51.173 (true trace 51.334)
Rayleigh ascent: R = 3.000000, x = [0.7071 0.7071]   (top eigenpair: 3, [0.7071 0.7071])
```

The output covers textbook Chapters 11 and 10:

- The log-density of the linear flow is that of Worked Example 11.1, and its gradient with respect to A and $\boldsymbol{b}$ (Exercise 11.5) passes the check. Trained by gradient descent, the flow ends with AA^{T} equal to the sample covariance and $\boldsymbol{b}$ equal to the sample mean — maximum likelihood, reached by differentiating a log-determinant.
- For a coupling layer, the formula $\ln|\det J| = \sum_j s_j$ agrees with the log-determinant of the numerically computed Jacobian, whose upper-right block is exactly zero, and the inverse recovers the input.
- Hutchinson's estimator converges to the trace like $1/\sqrt{k}$ (Figure 6.1); Rademacher vectors give a smaller standard deviation than Gaussian ones, as textbook Exercise 11.4 predicts.
- Gradient ascent on the Rayleigh quotient, renormalising after each step, finds the top eigenpair $(3, \frac{1}{\sqrt{2}}(1,1))$ of Worked Example 10.2.

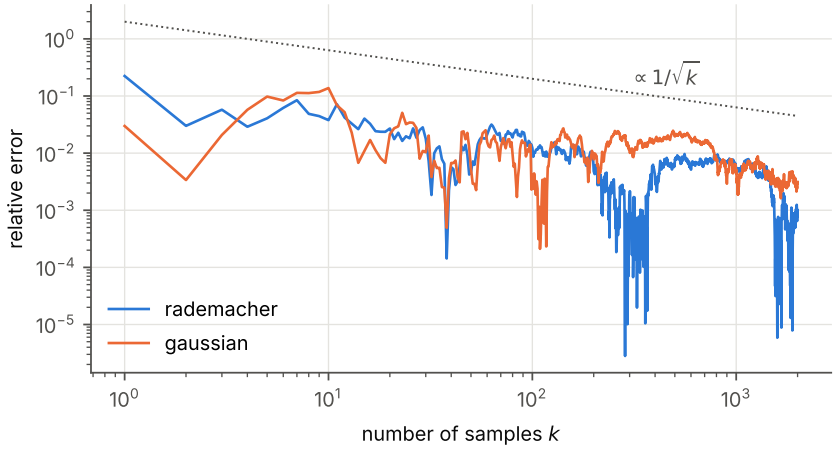


Figure 6.1: Relative error of the running Hutchinson estimate of $\operatorname{tr} M$ for a 50×50 positive definite M. Both decrease like $1/\sqrt{k}$; Rademacher vectors have the smaller constant.

Try it yourself.

1. Implement batch normalisation (standardise each column across the batch) with its backward pass, and check it. Show that, unlike layer norm, the gradient for one example depends on the other examples in the batch.
2. Stack four coupling layers, alternating which half is transformed, and train them by gradient descent on a two-moons data set. You will need the backward pass of the coupling layer: derive it with the differential method first.

Where Next: the Assignments

You have now seen every derivative of the course coded and checked. The Coding Assignments ask you to write them yourself behind a fixed interface, with an automatic test suite that compares every gradient with finite differences and checks the awkward cases: non-symmetric matrices, ReLU masks, broadcasting, causal masks, wide and tall shapes. Several functions there are not in this manual — the conditional Gaussian, the KL divergence between full-covariance Gaussians, batch normalisation, MSE and sigmoid layers, more Tensor operations — so the assignments are not a re-typing exercise. Attempt each one before re-reading the matching part.

APPENDIX A

Complete Source of `mcalc.py`

The whole library in one piece. Comment lines of the form `# [begin:...]` and `# [end:...]` mark the excerpts printed in the chapters.

```
"""
mcalc.py -- matrix calculus for deep learning, implemented in NumPy.

Companion code for the "NumPy Lab Manual".  Rules of the course:
  * NumPy only; numpy.linalg (solve, inv, det, slogdet, cholesky, eigh) is allowed;
  * nothing that differentiates for you (no JAX, PyTorch, TensorFlow, autograd):
    every gradient here is derived by hand and checked numerically.

Sections (match the parts of the Lab Manual):
    Part 1  numerical derivatives and gradient checking; functions of matrices; identities
    Part 2  vector derivatives; gradient descent and Newton's method
    Part 3  matrix derivatives; Gaussians
    Part 4  backpropagation: layers and a multilayer perceptron
    Part 5  automatic differentiation (dual numbers, a reverse-mode Tensor); stability
    Part 6  deep-learning layers: attention, layer norm, flows, Hutchinson, PCA
"""
import numpy as np


# ======================================================================
# Part 1 -- numerical derivatives
# ======================================================================

# [begin:numgrad]
def num_grad(f, X, h=1e-6):
    """Central-difference gradient of a scalar function f at X (any shape).

    Entry i of the result is (f(X + h e_i) - f(X - h e_i)) / 2h, so the
    result has the shape of X -- the rule 'the gradient has the shape of
    the variable'.  Costs 2 * X.size evaluations of f.
    """
    X = np.array(X, dtype=float)                # a private copy we can perturb
    G = np.zeros_like(X)
    for idx in np.ndindex(X.shape):
        old = X[idx]
        X[idx] = old + h
        f_plus = f(X)
        X[idx] = old - h
        f_minus = f(X)
        X[idx] = old                            # restore
        G[idx] = (f_plus - f_minus) / (2 * h)
    return G


def num_jacobian(f, x, h=1e-6):
    """Central-difference Jacobian J[i, j] = d f_i / d x_j of f: R^n -> R^m.

```

```
    Inputs and outputs of any shape are flattened, so J is (m, n).
    """
    x = np.array(x, dtype=float)
    m = np.size(f(x))
    J = np.zeros((m, x.size))
    for j, idx in enumerate(np.ndindex(x.shape)):
        old = x[idx]
        x[idx] = old + h
        f_plus = np.ravel(f(x))
        x[idx] = old - h
        f_minus = np.ravel(f(x))
        x[idx] = old
        J[:, j] = (f_plus - f_minus) / (2 * h)
    return J
# [end:numgrad]


# [begin:check]
def rel_error(a, b, tiny=1e-12):
    """Relative error ||a - b|| / max(||a||, ||b||, tiny) (Frobenius norms)."""
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    return float(np.linalg.norm(a - b) / max(np.linalg.norm(a), np.linalg.norm(b), tiny))


def check_grad(f, grad, X, h=1e-6):
    """Relative error between an analytic gradient and central differences."""
    return rel_error(grad, num_grad(f, X, h))


def check_directional(f, grad, X, h=1e-6, seed=0):
    """Cheap check along one random direction v: two evaluations of f.

    Compares (f(X + h v) - f(X - h v)) / 2h with <grad, v>.
    """
    X = np.asarray(X, dtype=float)
    v = np.random.default_rng(seed).standard_normal(X.shape)
    fd = (f(X + h * v) - f(X - h * v)) / (2 * h)
    an = float(np.sum(grad * v))
    return abs(fd - an) / max(abs(fd), abs(an), 1e-12)


def complex_step_grad(f, x, h=1e-20):
    """Complex-step gradient Im f(x + i h e_j) / h: no subtraction, full accuracy.

    Only valid for f built from analytic operations (no abs, max, comparisons).
    """
    x = np.array(x, dtype=complex)
    g = np.zeros(x.shape)
    for idx in np.ndindex(x.shape):
        old = x[idx]
        x[idx] = old + 1j * h
        g[idx] = np.imag(f(x)) / h
        x[idx] = old
    return g
# [end:check]


# ============================================================================
# Part 1 -- functions of matrices and identities
```

```
# ====================================================================

# [begin:matfun]
def logdet_chol(S):
    """ln|S| for symmetric positive definite S, as 2 * sum(log diag(L)), S = L L^T."""
    L = np.linalg.cholesky(S)
    return 2.0 * float(np.sum(np.log(np.diag(L))))


def sym_fn(S, fn):
    """Spectral function f(S) = Q f(Lambda) Q^T of a symmetric matrix."""
    w, Q = np.linalg.eigh(S)
    return (Q * fn(w)) @ Q.T                  # Q diag(f(w)) Q^T without building diag


def sqrtm_sym(S):
    return sym_fn(S, lambda w: np.sqrt(np.clip(w, 0.0, None)))


def inv_sqrtm_sym(S):
    return sym_fn(S, lambda w: 1.0 / np.sqrt(w))


def expm_sym(S):
    return sym_fn(S, np.exp)


def logm_sym(S):
    return sym_fn(S, np.log)


def expm_taylor(A, terms=20):
    """exp(A) for any square A: Taylor series with scaling and squaring.

    exp(A) = exp(A / 2^s)^(2^s); choosing s so that ||A / 2^s|| < 1/2 makes
    the series converge fast, then s squarings undo the scaling.
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    norm1 = np.max(np.sum(np.abs(A), axis=0)) if A.size else 0.0
    s = max(0, int(np.ceil(np.log2(norm1))) + 1) if norm1 > 0 else 0
    B = A / 2.0 ** s
    E, term = np.eye(n), np.eye(n)
    for k in range(1, terms + 1):
        term = term @ B / k
        E = E + term
    for _ in range(s):
        E = E @ E
    return E
# [end:matfun]


# [begin:identities]
def sherman_morrison(Ainv, u, v):
    """(A + u v^T)^{-1} from A^{-1} in O(n^2)."""
    Au = Ainv @ u
    vA = v @ Ainv
    return Ainv - np.outer(Au, vA) / (1.0 + v @ Au)

```

```
166
167 def woodbury(Ainv, U, C, V):
168     """(A + U C V)^{-1} from A^{-1}: only a k x k system is solved."""
169     K = np.linalg.inv(C) + V @ Ainv @ U          # k x k
170     return Ainv - Ainv @ U @ np.linalg.solve(K, V @ Ainv)
171
172
173 def block_inverse(A, B, C, D):
174     """Inverse of [[A, B], [C, D]] via the Schur complement of D."""
175     Dinv = np.linalg.inv(D)
176     P = np.linalg.inv(A - B @ Dinv @ C)          # (M/D)^{-1}
177     top = np.hstack([P, -P @ B @ Dinv])
178     bottom = np.hstack([-Dinv @ C @ P, Dinv + Dinv @ C @ P @ B @ Dinv])
179     return np.vstack([top, bottom])
180 # [end:identities]
181
182
183 # ======================================================================
184 # Part 2 -- vector derivatives and optimisation
185 # ======================================================================
186
187 # [begin:vecfun]
188 def quadratic(x, A, b):
189     """f = x^T A x + b^T x with gradient (A + A^T) x + b and Hessian A + A^T."""
190     f = x @ A @ x + b @ x
191     return f, (A + A.T) @ x + b, A + A.T
192
193
194 def least_squares(w, Phi, t, lam=0.0):
195     """E = 1/2 ||Phi w - t||^2 + lam/2 ||w||^2, its gradient and Hessian."""
196     r = Phi @ w - t
197     E = 0.5 * r @ r + 0.5 * lam * w @ w
198     g = Phi.T @ r + lam * w
199     H = Phi.T @ Phi + lam * np.eye(len(w))
200     return E, g, H
201 # [end:vecfun]
202
203
204 # [begin:stable]
205 def logsumexp(a, axis=-1, keepdims=False):
206     """Stable log(sum(exp(a))): subtract the maximum first."""
207     a = np.asarray(a, dtype=float)
208     m = np.max(a, axis=axis, keepdims=True)
209     out = m + np.log(np.sum(np.exp(a - m), axis=axis, keepdims=True))
210     return out if keepdims else np.squeeze(out, axis=axis)
211
212
213 def softmax(a, axis=-1):
214     """Stable softmax along an axis."""
215     a = np.asarray(a, dtype=float)
216     e = np.exp(a - np.max(a, axis=axis, keepdims=True))
217     return e / np.sum(e, axis=axis, keepdims=True)
218
219
220 def softplus(a):
221     """log(1 + exp(a)) without overflow: max(a, 0) + log1p(exp(-|a|))."""
222     a = np.asarray(a, dtype=float)
223     return np.maximum(a, 0.0) + np.log1p(np.exp(-np.abs(a)))
224
```

```
225
226 def sigmoid(a):
227     """Logistic sigmoid, evaluated so that exp never overflows."""
228     a = np.asarray(a, dtype=float)
229     e = np.exp(-np.abs(a))                      # always <= 1
230     return np.where(a >= 0, 1.0 / (1.0 + e), e / (1.0 + e))
231
232
233 def log_sigmoid(a):
234     return -softplus(-np.asarray(a, dtype=float))
235 # [end:stable]
236
237
238 # [begin:vecjac]
239 def softmax_jacobian(a):
240     """d softmax(a) / d a = diag(s) - s s^T."""
241     s = softmax(a)
242     return np.diag(s) - np.outer(s, s)
243
244
245 def logistic_loss(w, Phi, t):
246     """Cross-entropy of logistic regression: value, gradient Phi^T (y - t), Hessian Phi^T R Phi.
247
248     The value is computed as sum(softplus(a) - t a), which never takes log(0).
249     """
250     a = Phi @ w
251     E = float(np.sum(softplus(a) - t * a))
252     y = sigmoid(a)
253     g = Phi.T @ (y - t)
254     H = Phi.T @ (Phi * (y * (1 - y))[:, None])
255     return E, g, H
256 # [end:vecjac]
257
258
259 # [begin:optim]
260 def gradient_descent(grad_fn, x0, lr, n_steps):
261     """Plain gradient descent; returns the array of iterates x_0, ..., x_n."""
262     xs = [np.array(x0, dtype=float)]
263     for _ in range(n_steps):
264         xs.append(xs[-1] - lr * grad_fn(xs[-1]))
265     return np.array(xs)
266
267
268 def newton(grad_fn, hess_fn, x0, n_steps):
269     """Newton's method x <- x - H^{-1} g (a solve, never an inverse)."""
270     xs = [np.array(x0, dtype=float)]
271     for _ in range(n_steps):
272         x = xs[-1]
273         xs.append(x - np.linalg.solve(np.atleast_2d(hess_fn(x)), np.atleast_1d(grad_fn(x))))
274     return np.array(xs)
275 # [end:optim]
276
277
278 # ======================================================================
279 # Part 3 -- matrix derivatives and Gaussians
280 # ======================================================================
281
282 # [begin:catalogue]
283 # Each entry: name -> (f(X, P), grad(X, P)), where P holds the constants.
```

```
CATALOGUE = {
    "tr(A X)":          (lambda X, P: np.trace(P["A"] @ X),
                         lambda X, P: P["A"].T),
    "tr(A X B)":        (lambda X, P: np.trace(P["A"] @ X @ P["B"]),
                         lambda X, P: P["A"].T @ P["B"].T),
    "tr(X^T A X)":      (lambda X, P: np.trace(X.T @ P["A"] @ X),
                         lambda X, P: (P["A"] + P["A"].T) @ X),
    "a^T X b":          (lambda X, P: P["a"] @ X @ P["b"],
                         lambda X, P: np.outer(P["a"], P["b"])),
    "ln|det X|":        (lambda X, P: np.linalg.slogdet(X)[1],
                         lambda X, P: np.linalg.inv(X).T),
    "det X":            (lambda X, P: np.linalg.det(X),
                         lambda X, P: np.linalg.det(X) * np.linalg.inv(X).T),
    "tr(X^-1 A)":       (lambda X, P: np.trace(np.linalg.solve(X, P["A"])),
                         lambda X, P: -np.linalg.inv(X).T @ P["A"].T @ np.linalg.inv(X).T),
    "a^T X^-1 b":       (lambda X, P: P["a"] @ np.linalg.solve(X, P["b"]),
                         lambda X, P: -np.linalg.inv(X).T @ np.outer(P["a"], P["b"]) @
    ↪ np.linalg.inv(X).T),
    "||A X - B||_F^2":  (lambda X, P: np.sum((P["A"] @ X - P["B"]) ** 2),
                         lambda X, P: 2 * P["A"].T @ (P["A"] @ X - P["B"])),
    "tr(A^T tanh(X))":  (lambda X, P: np.sum(P["A"] * np.tanh(X)),
                         lambda X, P: P["A"] * (1 - np.tanh(X) ** 2)),
}
# [end:catalogue]


# [begin:gauss]
def gaussian_nll(X, mu, Sigma):
    """Negative log-likelihood of the rows of X under N(mu, Sigma), with gradients.

    Returns (value, d/dmu, d/dSigma) where
        d/dmu    = -Sigma^{-1} sum_n (x_n - mu)
        d/dSigma = 1/2 (N Sigma^{-1} - Sigma^{-1} S Sigma^{-1}),  S = sum (x_n-mu)(x_n-mu)^T
    """
    X = np.atleast_2d(X)
    N, D = X.shape
    diff = X - mu
    Sinv = np.linalg.inv(Sigma)
    S = diff.T @ diff
    # slogdet, not logdet_chol: Cholesky reads only the lower triangle, so a finite
    # difference that perturbs an upper entry would see a different function
    value = 0.5 * N * D * np.log(2 * np.pi) + 0.5 * N * np.linalg.slogdet(Sigma)[1] + 0.5 *
    ↪ np.sum(Sinv * S)
    dmu = -Sinv @ diff.sum(axis=0)
    dSigma = 0.5 * (N * Sinv - Sinv @ S @ Sinv)
    return float(value), dmu, dSigma


def gaussian_mle(X):
    """Maximum-likelihood mean and covariance (divide by N, as in Bishop)."""
    X = np.atleast_2d(X)
    mu = X.mean(axis=0)
    diff = X - mu
    return mu, diff.T @ diff / X.shape[0]


def fit_gaussian_gd(X, lr=0.1, steps=500):
    """Fit N(mu, L L^T) by gradient descent on (mu, L), L lower triangular.

```

```
    Chain rule through Sigma = L L^T:  dE/dL = 2 (dE/dSigma) L, keeping the
    lower triangle.  Returns mu, Sigma and the loss history (NLL per point).
    """
    X = np.atleast_2d(X)
    N, D = X.shape
    mu, L = np.zeros(D), np.eye(D)
    history = []
    for _ in range(steps):
        E, dmu, dSigma = gaussian_nll(X, mu, L @ L.T)
        history.append(E / N)
        mu = mu - lr * dmu / N
        L = L - lr * np.tril(2 * dSigma @ L) / N
    return mu, L @ L.T, np.array(history)


def kl_diag_standard(mu, logvar):
    """KL( N(mu, diag(exp(logvar))) || N(0, I) ) and its gradients (the VAE term)."""
    var = np.exp(logvar)
    value = 0.5 * float(np.sum(var + mu ** 2 - 1.0 - logvar))
    return value, mu.copy(), 0.5 * (var - 1.0)
# [end:gauss]


# ====================================================================
# Part 4 -- backpropagation
# ====================================================================

# [begin:layers]
class Linear:
    """Y = X W + b.   backward:  dW = X^T dY,  db = sum_rows(dY),  dX = dY W^T."""

    def __init__(self, d_in, d_out, rng, scale=None):
        scale = np.sqrt(1.0 / d_in) if scale is None else scale
        self.W = rng.standard_normal((d_in, d_out)) * scale
        self.b = np.zeros(d_out)

    def forward(self, X):
        self.X = X                                  # keep the input for backward
        return X @ self.W + self.b

    def backward(self, dY):
        self.dW = self.X.T @ dY
        self.db = dY.sum(axis=0)
        return dY @ self.W.T

    def params(self):
        return [self.W, self.b]

    def grads(self):
        return [self.dW, self.db]


class Tanh:
    """H = tanh(Z).   backward:  dZ = dH * (1 - H^2)."""

    def forward(self, Z):
        self.H = np.tanh(Z)
        return self.H

```

```
    def backward(self, dH):
        return dH * (1.0 - self.H ** 2)

    def params(self):
        return []

    def grads(self):
        return []


class ReLU:
    """H = max(Z, 0).   backward:  dZ = dH * [Z > 0]."""

    def forward(self, Z):
        self.mask = Z > 0
        return Z * self.mask

    def backward(self, dH):
        return dH * self.mask

    def params(self):
        return []

    def grads(self):
        return []


class SoftmaxCrossEntropy:
    """Mean cross-entropy of softmax(A) with integer labels.   backward: (S - T) / N."""

    def forward(self, A, y):
        N = A.shape[0]
        logp = A - logsumexp(A, axis=1, keepdims=True)    # stable log-softmax
        self.S, self.y = np.exp(logp), y
        return float(-np.mean(logp[np.arange(N), y]))

    def backward(self):
        N = self.S.shape[0]
        G = self.S.copy()
        G[np.arange(N), self.y] -= 1.0                    # S - T for one-hot T
        return G / N
# [end:layers]


# [begin:mlp]
class MLP:
    """A multilayer perceptron with a hand-written backward pass."""

    def __init__(self, sizes, activation="tanh", seed=0):
        rng = np.random.default_rng(seed)
        act = {"tanh": Tanh, "relu": ReLU}[activation]
        self.layers = []
        for i, (d_in, d_out) in enumerate(zip(sizes[:-1], sizes[1:])):
            self.layers.append(Linear(d_in, d_out, rng))
            if i < len(sizes) - 2:                        # no activation after the last layer
                self.layers.append(act())
        self.loss_fn = SoftmaxCrossEntropy()

    def forward(self, X):
```

```
        for layer in self.layers:
            X = layer.forward(X)
        return X                                            # logits

    def loss(self, X, y):
        return self.loss_fn.forward(self.forward(X), y)

    def backward(self):
        g = self.loss_fn.backward()
        for layer in reversed(self.layers):
            g = layer.backward(g)
        return g                                            # gradient w.r.t. the input

    def params(self):
        return [p for layer in self.layers for p in layer.params()]

    def grads(self):
        return [g for layer in self.layers for g in layer.grads()]


def make_spirals(n_per_class=100, n_classes=3, noise=0.2, seed=0):
    """The classic 2-D spiral classification problem."""
    rng = np.random.default_rng(seed)
    X, y = [], []
    for k in range(n_classes):
        r = np.linspace(0.0, 1.0, n_per_class)
        theta = np.linspace(k * 4.0, (k + 1) * 4.0, n_per_class) +
    ↪ rng.standard_normal(n_per_class) * noise
        X.append(np.column_stack([r * np.sin(theta), r * np.cos(theta)]))
        y.append(np.full(n_per_class, k))
    return np.vstack(X), np.concatenate(y)


def train(model, X, y, lr=0.5, epochs=2000, momentum=0.9):
    """Full-batch gradient descent with momentum. Returns the loss history."""
    velocity = [np.zeros_like(p) for p in model.params()]
    history = []
    for _ in range(epochs):
        history.append(model.loss(X, y))
        model.backward()
        for p, g, v in zip(model.params(), model.grads(), velocity):
            v *= momentum
            v -= lr * g
            p += v                                          # in-place: updates the layer
    return np.array(history)
# [end:mlp]


# ====================================================================
# Part 5 -- automatic differentiation
# ====================================================================

# [begin:dual]
class Dual:
    """Dual number a + b eps with eps^2 = 0: forward-mode automatic differentiation."""

    def __init__(self, val, der=0.0):
        self.val, self.der = float(val), float(der)

```

```
    @staticmethod
    def _lift(x):
        return x if isinstance(x, Dual) else Dual(x, 0.0)

    def __add__(self, o):
        o = Dual._lift(o)
        return Dual(self.val + o.val, self.der + o.der)
    __radd__ = __add__

    def __sub__(self, o):
        o = Dual._lift(o)
        return Dual(self.val - o.val, self.der - o.der)

    def __rsub__(self, o):
        return Dual._lift(o) - self

    def __mul__(self, o):
        o = Dual._lift(o)
        return Dual(self.val * o.val, self.val * o.der + self.der * o.val)
    __rmul__ = __mul__

    def __truediv__(self, o):
        o = Dual._lift(o)
        return Dual(self.val / o.val, (self.der * o.val - self.val * o.der) / o.val ** 2)

    def __rtruediv__(self, o):
        return Dual._lift(o) / self

    def __neg__(self):
        return Dual(-self.val, -self.der)

    def __pow__(self, p):                    # real exponent p
        return Dual(self.val ** p, p * self.val ** (p - 1) * self.der)

    def __repr__(self):
        return f"Dual({self.val:.6g}, {self.der:.6g})"


def _unary(fn, dfn):
    def op(x):
        if isinstance(x, Dual):
            return Dual(fn(x.val), dfn(x.val) * x.der)
        return fn(x)
    return op


sin = _unary(np.sin, np.cos)
cos = _unary(np.cos, lambda v: -np.sin(v))
exp = _unary(np.exp, np.exp)
log = _unary(np.log, lambda v: 1.0 / v)
tanh = _unary(np.tanh, lambda v: 1.0 - np.tanh(v) ** 2)


def derivative(f, x):
    """f'(x) by evaluating f on the dual number x + 1 eps."""
    return f(Dual(x, 1.0)).der
# [end:dual]


```

```python
# [begin:tensor]
def _unbroadcast(grad, shape):
    """Sum a gradient over the axes that broadcasting expanded, back to `shape`."""
    while grad.ndim > len(shape):
        grad = grad.sum(axis=0)
    for axis, n in enumerate(shape):
        if n == 1 and grad.shape[axis] != 1:
            grad = grad.sum(axis=axis, keepdims=True)
    return grad


class Tensor:
    """A NumPy array that records operations, for reverse-mode differentiation."""

    def __init__(self, data, parents=(), op=""):
        self.data = np.asarray(data, dtype=float)
        self.grad = np.zeros_like(self.data)
        self._parents = parents
        self._backward = lambda: None
        self.op = op

    @property
    def shape(self):
        return self.data.shape

    @staticmethod
    def _lift(x):
        return x if isinstance(x, Tensor) else Tensor(x)

    # ---- elementary operations: each defines the forward value and its backward rule
    def __add__(self, other):
        other = Tensor._lift(other)
        out = Tensor(self.data + other.data, (self, other), "+")
        def _backward():
            self.grad += _unbroadcast(out.grad, self.shape)
            other.grad += _unbroadcast(out.grad, other.shape)
        out._backward = _backward
        return out
    __radd__ = __add__

    def __mul__(self, other):
        other = Tensor._lift(other)
        out = Tensor(self.data * other.data, (self, other), "*")
        def _backward():
            self.grad += _unbroadcast(out.grad * other.data, self.shape)
            other.grad += _unbroadcast(out.grad * self.data, other.shape)
        out._backward = _backward
        return out
    __rmul__ = __mul__

    def __neg__(self):
        return self * -1.0

    def __sub__(self, other):
        return self + (-Tensor._lift(other))

    def __rsub__(self, other):
        return Tensor._lift(other) + (-self)

```

```
    def __pow__(self, p):                          # real exponent p
        out = Tensor(self.data ** p, (self,), f"**{p}")
        def _backward():
            self.grad += out.grad * p * self.data ** (p - 1)
        out._backward = _backward
        return out

    def __truediv__(self, other):
        return self * Tensor._lift(other) ** -1.0

    def __matmul__(self, other):                   # 2-D matrices
        other = Tensor._lift(other)
        out = Tensor(self.data @ other.data, (self, other), "@")
        def _backward():
            self.grad += out.grad @ other.data.T          # dX = dY W^T
            other.grad += self.data.T @ out.grad          # dW = X^T dY
        out._backward = _backward
        return out

    @property
    def T(self):
        out = Tensor(self.data.T, (self,), "T")
        def _backward():
            self.grad += out.grad.T
        out._backward = _backward
        return out

    def sum(self, axis=None, keepdims=False):
        out = Tensor(self.data.sum(axis=axis, keepdims=keepdims), (self,), "sum")
        def _backward():
            g = out.grad
            if axis is not None and not keepdims:
                g = np.expand_dims(g, axis)
            self.grad += np.broadcast_to(g, self.shape)
        out._backward = _backward
        return out

    def mean(self, axis=None):
        n = self.data.size if axis is None else self.data.shape[axis]
        return self.sum(axis=axis) * (1.0 / n)

    def _elementwise(self, value, local_grad, op):
        out = Tensor(value, (self,), op)
        def _backward():
            self.grad += out.grad * local_grad
        out._backward = _backward
        return out

    def exp(self):
        e = np.exp(self.data)
        return self._elementwise(e, e, "exp")

    def log(self):
        return self._elementwise(np.log(self.data), 1.0 / self.data, "log")

    def tanh(self):
        t = np.tanh(self.data)
        return self._elementwise(t, 1.0 - t ** 2, "tanh")

```

```
    def relu(self):
        return self._elementwise(np.maximum(self.data, 0.0), (self.data > 0).astype(float), "relu")

    def sigmoid(self):
        s = sigmoid(self.data)
        return self._elementwise(s, s * (1.0 - s), "sigmoid")

    # ---- the reverse sweep
    def backward(self):
        """Fill .grad of every tensor this one depends on (self must be a scalar)."""
        order, seen = [], set()
        def visit(v):                           # depth-first topological sort
            if id(v) not in seen:
                seen.add(id(v))
                for p in v._parents:
                    visit(p)
                order.append(v)
        visit(self)
        self.grad = np.ones_like(self.data)     # seed: dL/dL = 1
        for v in reversed(order):
            v._backward()


def cross_entropy(logits, y):
    """Mean softmax cross-entropy as ONE primitive: stable forward, (S - T)/N backward."""
    N = logits.shape[0]
    logp = logits.data - logsumexp(logits.data, axis=1, keepdims=True)
    out = Tensor(-np.mean(logp[np.arange(N), y]), (logits,), "xent")
    def _backward():
        G = np.exp(logp)
        G[np.arange(N), y] -= 1.0
        logits.grad += out.grad * G / N
    out._backward = _backward
    return out


def zero_grad(params):
    for p in params:
        p.grad = np.zeros_like(p.data)
# [end:tensor]


# ======================================================================
# Part 6 -- deep-learning layers
# ======================================================================

# [begin:attention]
def softmax_rows_backward(A, dA):
    """Backward pass of a row-wise softmax: dS = A * (dA - rowsum(dA * A))."""
    return A * (dA - np.sum(dA * A, axis=1, keepdims=True))


def attention_forward(X, Wq, Wk, Wv, causal=False):
    """One self-attention head. Returns Y and a cache for the backward pass."""
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    d_k = Wq.shape[1]
    S = Q @ K.T / np.sqrt(d_k)
    if causal:                                  # token i may only attend to j <= i
        S = np.where(np.tril(np.ones_like(S, dtype=bool)), S, -np.inf)
```

```
    A = softmax(S, axis=1)
    Y = A @ V
    return Y, (X, Wq, Wk, Wv, Q, K, V, A, d_k)


def attention_backward(dY, cache):
    """Gradients of self-attention: returns dX, dWq, dWk, dWv."""
    X, Wq, Wk, Wv, Q, K, V, A, d_k = cache
    dV = A.T @ dY
    dA = dY @ V.T
    dS = softmax_rows_backward(A, dA)
    dQ = dS @ K / np.sqrt(d_k)
    dK = dS.T @ Q / np.sqrt(d_k)
    dWq, dWk, dWv = X.T @ dQ, X.T @ dK, X.T @ dV
    dX = dQ @ Wq.T + dK @ Wk.T + dV @ Wv.T         # X is used three times: add
    return dX, dWq, dWk, dWv
# [end:attention]


# [begin:layernorm]
def layernorm_forward(X, gamma, beta, eps=1e-5):
    """Normalise each row of X, then scale and shift."""
    mu = X.mean(axis=1, keepdims=True)
    var = X.var(axis=1, keepdims=True)
    sig = np.sqrt(var + eps)
    Xh = (X - mu) / sig
    return gamma * Xh + beta, (Xh, sig, gamma)


def layernorm_backward(dY, cache):
    """dX = (dXh - mean(dXh) - Xh mean(dXh * Xh)) / sigma, row by row."""
    Xh, sig, gamma = cache
    dXh = dY * gamma
    dX = (dXh - dXh.mean(axis=1, keepdims=True)
          - Xh * (dXh * Xh).mean(axis=1, keepdims=True)) / sig
    return dX, (dY * Xh).sum(axis=0), dY.sum(axis=0)
# [end:layernorm]


# [begin:flows]
def coupling_forward(x, Ws, bs, Wt, bt):
    """Affine coupling layer on the rows of x (N x D, D even).

    z_A = x_A,  z_B = x_B * exp(s) + t,  s = tanh(x_A Ws + bs),  t = x_A Wt + bt.
    Returns z and log|det J| = sum(s) for every row.
    """
    d = x.shape[1] // 2
    xa, xb = x[:, :d], x[:, d:]
    s = np.tanh(xa @ Ws + bs)
    t = xa @ Wt + bt
    return np.hstack([xa, xb * np.exp(s) + t]), s.sum(axis=1)


def coupling_inverse(z, Ws, bs, Wt, bt):
    d = z.shape[1] // 2
    za, zb = z[:, :d], z[:, d:]
    s = np.tanh(za @ Ws + bs)
    t = za @ Wt + bt
    return np.hstack([za, (zb - t) * np.exp(-s)])
```

```

def linear_flow_nll(X, A, b):
    """Mean NLL of x = A z + b, z ~ N(0, I), with gradients w.r.t. A and b.

    dA = A^{-T} (I - mean(z z^T)),   db = -A^{-T} mean(z).
    """
    N, D = X.shape
    Z = np.linalg.solve(A, (X - b).T).T            # rows z_n = A^{-1}(x_n - b)
    value = 0.5 * D * np.log(2 * np.pi) + 0.5 * np.mean(np.sum(Z ** 2, axis=1)) \
        + np.linalg.slogdet(A)[1]
    AinvT = np.linalg.inv(A).T
    dA = AinvT @ (np.eye(D) - Z.T @ Z / N)
    db = -AinvT @ Z.mean(axis=0)
    return float(value), dA, db


def hutchinson_samples(matvec, n, n_samples, rng, kind="rademacher"):
    """Individual Hutchinson samples eps^T A eps (their mean estimates tr A)."""
    out = np.empty(n_samples)
    for i in range(n_samples):
        eps = rng.choice([-1.0, 1.0], size=n) if kind == "rademacher" else rng.standard_normal(n)
        out[i] = eps @ matvec(eps)
    return out


def rayleigh_quotient(S, x):
    """R(x) = x^T S x / x^T x and its gradient 2 (S x - R x) / x^T x."""
    xx = x @ x
    R = x @ S @ x / xx
    return float(R), 2.0 * (S @ x - R * x) / xx
# [end:flows]
```

APPENDIX B

Helper Scripts

plotstyle.py

```python
"""Shared Matplotlib style for the Lab Manual figures (import it before plotting)."""
import os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# categorical colours, always used in this order
C1, C2, C3 = "#2a78d6", "#eb6834", "#1baf7a"
INK, INK2, GRID = "#0b0b0b", "#52514e", "#e4e3df"

plt.rcParams.update({
    "figure.figsize": (6.0, 3.4),
    "figure.dpi": 150,
    "font.size": 9,
    "axes.edgecolor": INK2,
    "axes.labelcolor": INK,
    "axes.titlesize": 10,
    "axes.titleweight": "bold",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.color": GRID,
    "grid.linewidth": 0.6,
    "xtick.color": INK2,
    "ytick.color": INK2,
    "lines.linewidth": 1.6,
    "lines.markersize": 4.5,
    "legend.frameon": False,
    "savefig.bbox": "tight",
})


def savefig(name):
    """Save the current figure as figs/<name>.pdf and close it."""
    os.makedirs("figs", exist_ok=True)
    plt.savefig(os.path.join("figs", name + ".pdf"))
    plt.close()
```

run_demos.py

```
"""Run every demo script and capture its output in out/<name>.txt (figures go to figs/)."""
import glob, os, subprocess, sys, time
here = os.path.dirname(os.path.abspath(__file__))
os.makedirs(os.path.join(here, "out"), exist_ok=True)
env = dict(os.environ, PYTHONPATH=here)
for path in sorted(glob.glob(os.path.join(here, "demos", "d*.py"))):
    name = os.path.splitext(os.path.basename(path))[0]
    t0 = time.time()
    r = subprocess.run([sys.executable, path], cwd=here, env=env, capture_output=True, text=True)
    with open(os.path.join(here, "out", name + ".txt"), "w") as f:
        f.write(r.stdout.rstrip() + "\n")
    status = "ok" if r.returncode == 0 else "FAILED"
    print(f"{name:22s} {status}  ({time.time() - t0:.1f}s)")
    if r.returncode:
        print(r.stderr)
```